

GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

VOZES QUE
PENSAM: NLP,
FALA E EMOÇÃO
NA ERA DOS
ASSISTENTES
COGNITIVOS

LISTA DE FIGURAS

Figura 1 - Resultado do Código-fonte 1, sinal periódico senoidal	9
Figura 2 - Resultado do Código-fonte 2, sinal periódico retangular	11
Figura 3 - Resultado do Código-fonte 3, sinal periódico triangular	11
Figura 4 - Resultado do Código-fonte 4, espectrograma de sinais periódicos	12
Figura 5 - Interface do Audacity	18
Figura 6 - Gravando um áudio no Audacity	19
Figura 7 - Zoom no domínio do tempo	19
Figura 8 - Alterando a exibição do áudio no Audacity para espectrograma	20
Figura 9 - Espectrograma do áudio no Audacity	20
Figura 10 - Aplicando filtro de Graves e Agudos	21
Figura 11 - Removendo agudos no Audacity	21
Figura 12 - Espectrograma do áudio após filtragem de agudos	22
Figura 13 - Espectrograma do áudio antes e depois da remoção de 0,5s iniciais.....	23
Figura 14 - Exportando arquivo de áudio no Audacity.....	23
Figura 15 - Abordagens de uso de ML no processamento de áudio	25
Figura 16 - Features extraídas, saída do Código-fonte 5	29
Figura 17 - Dataset Vowel, saída do Código-fonte 6.....	31
Figura 18 - Resultados do reconhecedor de voz, saída do Código-fonte 7.....	33
Figura 19 - Combinando áudios com o Audacity	34
Figura 20 - Ajustando a duração de áudios combinados com o Audacity	34
Figura 21 - Espectrograma dos áudios combinados com o Audacity	34
Figura 22 - Exportando áudio combinado com o Audacity	35
Figura 23 - Resultado áudio combinado com o Audacity no domínio do tempo	35
Figura 24 - Resultado áudio combinado com o Audacity, vista de espectrograma ...	35
Figura 25 - Resultado áudio combinado filtrado com LPF no Python.....	37
Figura 26 - Resultado áudio filtrado com rede end-to-end	44
Figura 27 - Espectro audível humano e de outros animais	46
Figura 28 - Resultados do reconhecedor de vogais, saída do Código-fonte 14.....	50
Figura 29 - Diagrama de funcionamento de um ASR clássico	52
Figura 30 - Criando um ASR na IBM Cloud.....	54
Figura 31 - Instanciando o serviço de ASR/STT na IBM Cloud.....	54
Figura 32 - Serviço de ASR/STT na IBM Cloud pronto para ser usado	55
Figura 33 - Taxa de erro de transcrição, saída do Código-fonte 17	58
Figura 34 - Espectro de sons produzidos por humanos e de outros animais	59

LISTA DE QUADROS

Quadro 1 - Alguns tipos de arquivos de áudio.....	17
Quadro 2 - Elementos Linguísticos da Percepção da Fala	47
Quadro 3 - Unidades Linguísticas da Escrita e Transcrição Fonética	48

EMANIP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Sinal periódico senoidal	8
Código-fonte 2 - Sinal periódico de onda retangular	10
Código-fonte 3 - Sinal periódico de onda triangular	10
Código-fonte 4 - Plotando espectrograma de sinais periódicos	14
Código-fonte 5 - Extração de features de um sinal de áudio	29
Código-fonte 6 - Baixando dataset Vowel do OpenML	31
Código-fonte 7 - Treinando um reconhecedor de voz com o dataset Vowel	32
Código-fonte 8 - Aplicando filtro passa-baixa com Python	36
Código-fonte 9 - Gerando os espectrogramas da Figura 25	37
Código-fonte 10 - Preparação dos dados para rede end-to-end	40
Código-fonte 11 - Criação e treinamento da Rede Neural end-to-end	41
Código-fonte 12 - Inferência e reconstrução do áudio com a rede end-to-end	42
Código-fonte 13 - Geração dos gráficos de comparação para uso da rede end-to-end	44
Código-fonte 14 - Treinando um reconhecedor de vogais com o dataset Vowel	50
Código-fonte 15 - Uso de ASR pré-treinado do Hugging Face	53
Código-fonte 16 - Uso de ASR/STT da IBM Cloud	56
Código-fonte 17 - Calculando WER de dois ASR/STT	58
Código-fonte 18 - Criando um TTS concatenativo	62
Código-fonte 19 - Usando TTS da IBM Cloud	63

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Instalação do Librosa.....	28
Comando de prompt 2 - Bibliotecas do Hugging Face	53
Comando de prompt 3 - Biblioteca do STT e TTS da IBM Cloud	55
Comando de prompt 4 - Bibliotecas para cálculo de WER.....	56

EMENDAS

SUMÁRIO

1 VOZES QUE PENSAM: NLP, FALA E EMOÇÃO NA ERA DOS ASSISTENTES COGNITIVOS.....	7
1.1 Introdução ao processamento de sinais.....	7
1.1.1 O que é um sinal?	7
1.1.2 Propriedades básicas de sinais periódicos.....	7
1.1.3 Sinais analógicos e sinais digitais	14
1.1.4 Arquivos de áudio.....	16
1.1.5 Audacity.....	17
2 APRENDIZADO DE MÁQUINA NO PROCESSAMENTO DE ÁUDIO	24
2.1 Usando aprendizado de máquina no processamento de áudio.....	24
2.2 Extração de features	27
2.3 Classificação de áudio por features	30
2.4 Remoção de ruído de áudio por sistema end-to-end	33
3 SISTEMAS DE FALA E VOZ.....	46
3.1 Como os seres humanos entendem som e linguagem?.....	46
3.2 Reconhecedores de Fala	48
3.2.1 ASR Clássico	49
3.2.2 Usando um ASR pré-treinado	52
3.3 Como os seres humanos produzem som e linguagem?.....	59
3.4 Sintetizadores de Fala.....	60
3.5 Aplicações avançadas.....	63
REFERÊNCIAS.....	65

1 VOZES QUE PENSAM: NLP, FALA E EMOÇÃO NA ERA DOS ASSISTENTES COGNITIVOS

1.1 Introdução ao processamento de sinais

1.1.1 O que é um sinal?

Um sinal, em termos técnicos, é qualquer função que representa a variação temporal, espacial ou espacial-temporal de uma grandeza física ou abstrata, carregando informação sobre o comportamento de um sistema. Formalmente, um sinal pode ser modelado como uma função $x(t)$, quando depende do tempo contínuo, ou $x[n]$, quando depende de um índice discreto, sendo capaz de descrever fenômenos como tensão elétrica, pressão sonora, intensidade luminosa ou valores simbólicos produzidos por sistemas digitais.

A utilidade fundamental de um sinal está na sua capacidade de codificar informação que pode ser analisada, transformada e interpretada por sistemas de processamento analógicos ou digitais, permitindo tarefas como detecção, transmissão, controle, classificação e diagnóstico. Do ponto de vista de processamento, sinais podem apresentar propriedades como amplitude, frequência, fase, energia e potência, e podem ser estacionários ou não, determinísticos ou aleatórios, contínuos ou discretos, cada categoria influenciando os métodos de análise aplicáveis.

1.1.2 Propriedades básicas de sinais periódicos

Sinais periódicos apresentam um conjunto de propriedades fundamentais que caracterizam seu comportamento repetitivo ao longo do tempo. As principais são:

- **Período T :** é o intervalo de tempo mínimo após o qual o sinal $s(t)$ se repete integralmente. Se $s(t)$ é periódico, então $s(t) = s(t + T)$ para todo t .
- **Frequência f :** indica quantas repetições do padrão ocorrem por segundo. É o inverso do período $f = \frac{1}{T}$. Também se utiliza a frequência angular $\omega = 2\pi f$ para representar a frequência de sinais periódicos.

- **Amplitude:** representa a magnitude máxima do sinal. Pode referir-se à amplitude de pico, pico a pico ou amplitude RMS, dependendo da aplicação.
- **Fase φ :** descreve o deslocamento horizontal do sinal em relação a uma referência temporal. A fase inicial φ_0 define onde o ciclo começa e é essencial para combinar ou comparar sinais periódicos.
- **Forma de onda:** além das grandezas anteriores, um sinal periódico possui um padrão estrutural definido (senoidal, quadrado, triangular, “dente de serra” etc.), que determina suas características espectrais.
- **Conteúdo espectral:** pelo teorema de Fourier, qualquer sinal periódico pode ser decomposto em uma soma de harmônicas (senos e cossenos). Assim, seu espectro é composto por frequências discretas múltiplas da fundamental.

De maneira geral, um sinal periódico simples pode ser descrito matematicamente como: $s(t) = A \sin(\omega t)$. Vamos construir esse sinal periódico na prática utilizando Python:

```
import numpy as np
import matplotlib.pyplot as plt

# Parâmetros do sinal
freq = 5          # frequência em Hz
fs = 1000         # taxa de amostragem em Hz
A = 2             # amplitude máxima do sinal
t = np.linspace(0, 1, fs)  # vetor de tempo de 0 a 1 segundo

sinal = A*np.sin(2 * np.pi * freq * t)

plt.figure(figsize=(10, 4))
plt.plot(t, sinal)
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Código-fonte 1 - Sinal periódico senoidal

Fonte: Elaborado pelo autor (2025)

O resultado da execução do código-fonte anterior é mostrado na figura “Resultado do Código-fonte 1, sinal periódico senoidal”. Observe como as propriedades básicas do sinal são codificadas e como elas podem ser identificadas

no gráfico: o pico de amplitude (positivo ou negativo) tem magnitude máxima de 2, o valor definido na variável $A=2$; a frequência é adicionada como argumento da função seno através de ω , fazendo-se $2 \cdot \pi \cdot f$; o formato do sinal é dito senoidal pois segue a função seno. Perceba que o período T da onda é de 0,2 segundos (quando a onda completa um ciclo) e que a amplitude do sinal começa em 0 para $t = 0$.

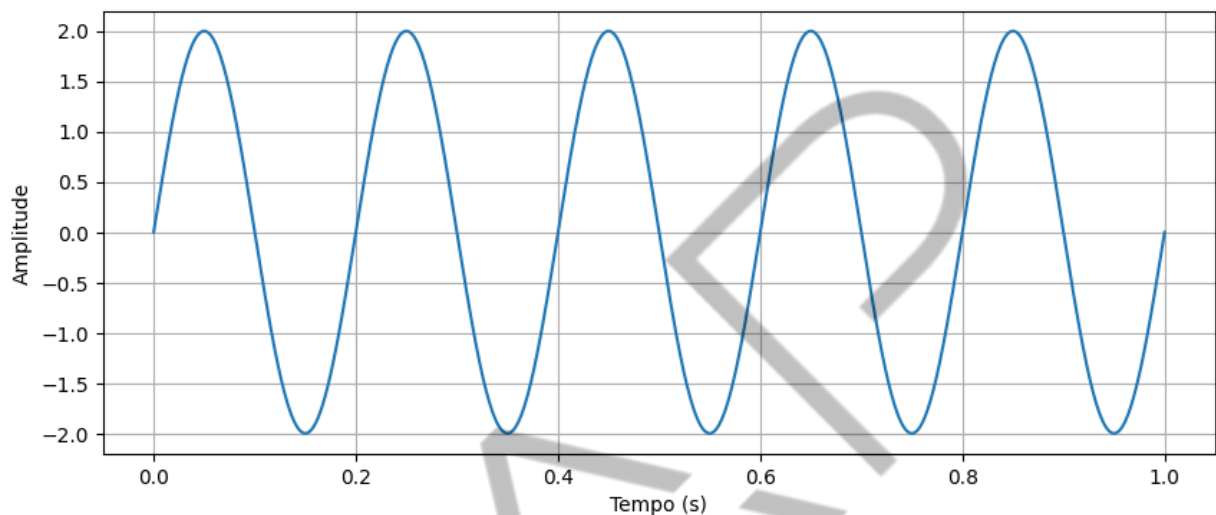


Figura 1 - Resultado do Código-fonte 1, sinal periódico senoidal
Fonte: Elaborado pelo autor (2025)

Se estamos interessados em criar uma onda quadrada (retangular) ou uma onda triangular, podemos utilizar um módulo de processamento de sinais da biblioteca SciPy para criar esses sinais. Observe os próximos códigos-fonte e seus respectivos resultados nas figuras apresentadas a seguir.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import square

# Parâmetros do sinal
freq = 5          # frequência em Hz
fs = 1000         # taxa de amostragem em Hz
A = 2            # amplitude máxima
t = np.linspace(0, 1, fs)  # vetor de tempo de 0 a 1
segundo

sinal = A * square(2 * np.pi * freq * t)

plt.figure(figsize=(10, 4))
plt.plot(t, sinal)
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Código-fonte 2 - Sinal periódico de onda retangular
Fonte: Elaborado pelo autor (2025)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import sawtooth

# Parâmetros do sinal
freq = 5          # frequência em Hz
fs = 1000         # taxa de amostragem em Hz
A = 2            # amplitude máxima
t = np.linspace(0, 1, fs)  # vetor de tempo de 0 a 1
segundo

sinal = A * sawtooth(2 * np.pi * freq * t, width=0.5)

plt.figure(figsize=(10, 4))
plt.plot(t, sinal)
plt.xlabel("Tempo (s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
```

Código-fonte 3 - Sinal periódico de onda triangular
Fonte: Elaborado pelo autor (2025)

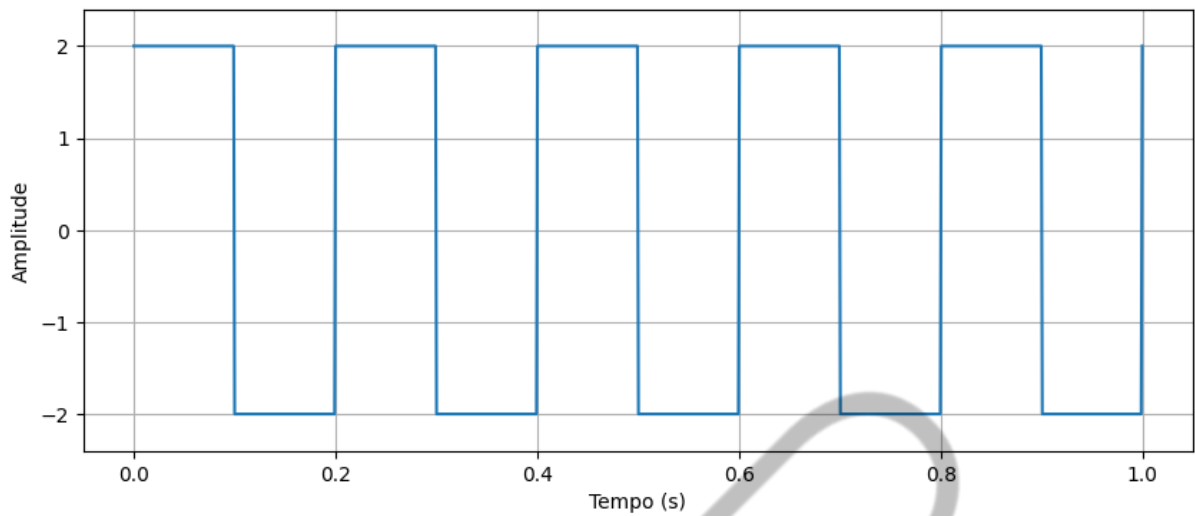


Figura 2 - Resultado do Código-fonte 2, sinal periódico retangular
Fonte: Elaborado pelo autor (2025)

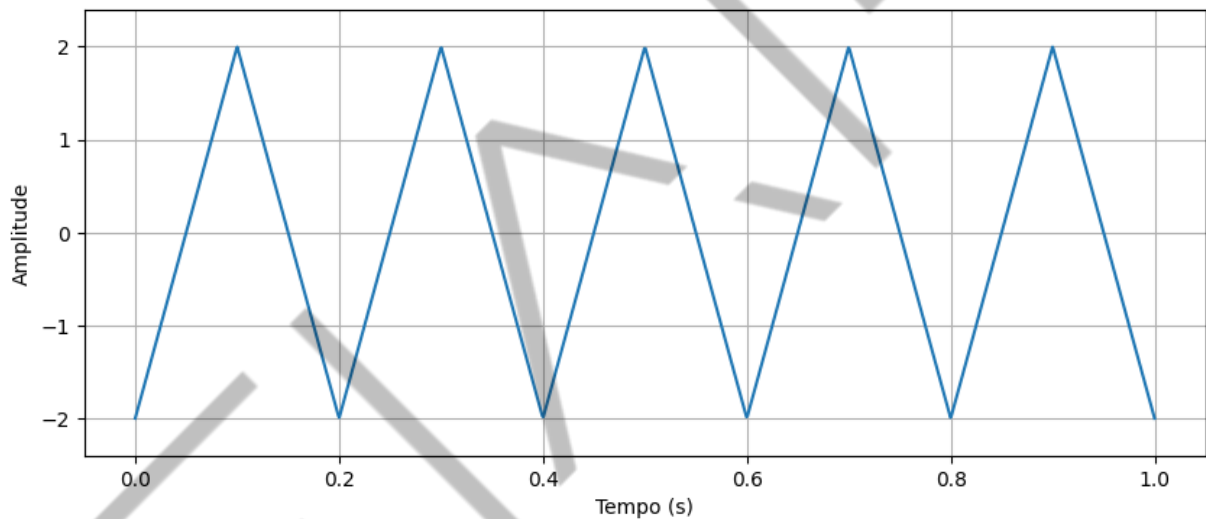


Figura 3 - Resultado do Código-fonte 3, sinal periódico triangular
Fonte: Elaborado pelo autor (2025)

Nos gráficos anteriores, vemos sinais periódicos com a mesma frequência, mas com formatos diferentes. Em todos os casos, estamos tratando do sinal no domínio do tempo, ou seja, como a amplitude da variável de interesse varia para cada instante no tempo. Também podemos observá-los no domínio da frequência. Veja o resultado:

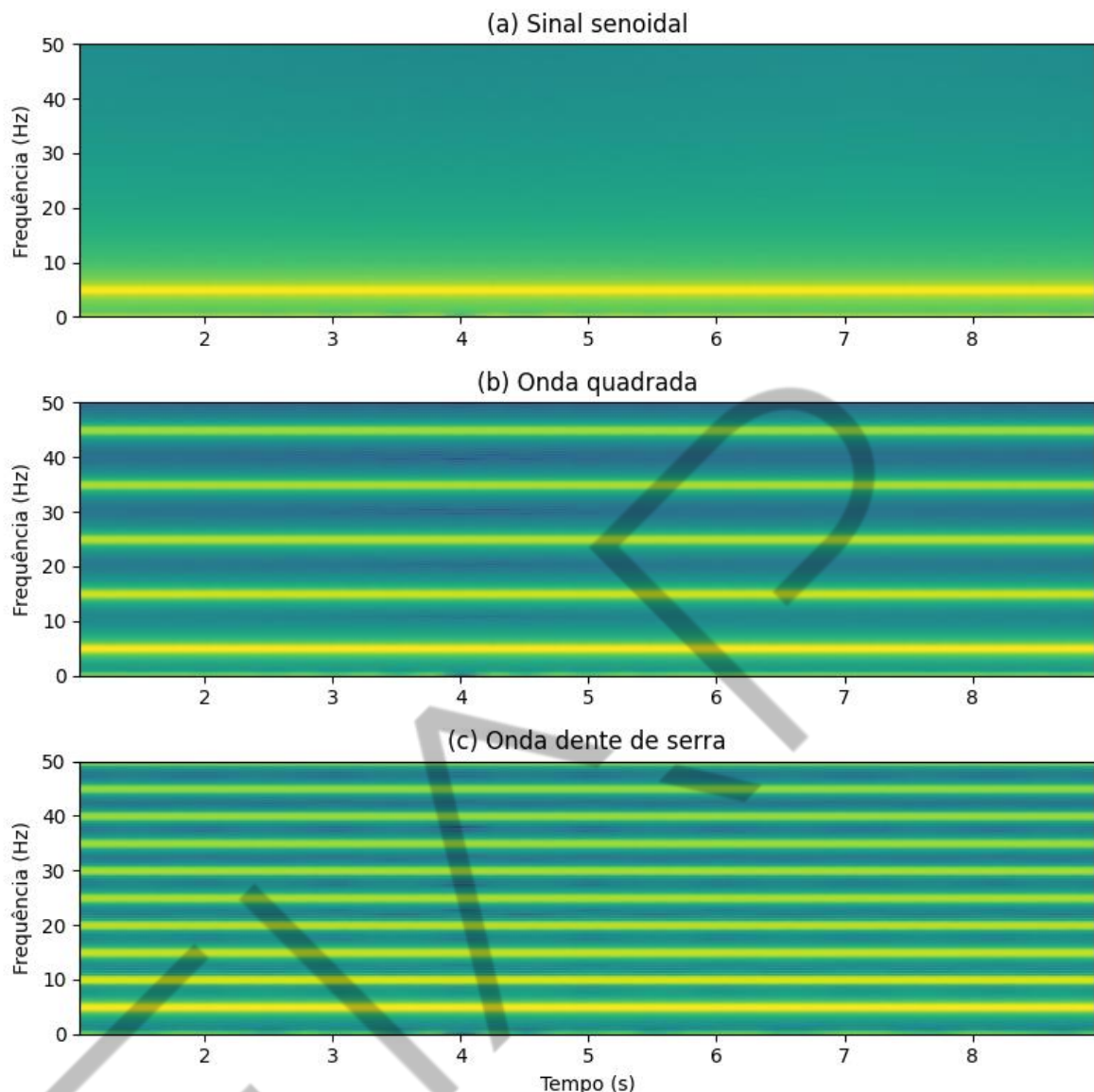


Figura 4 - Resultado do Código-fonte 4, espectrograma de sinais periódicos
Fonte: Elaborado pelo autor (2025)

Nos gráficos no domínio da frequência, mostrados na figura anterior, observa-se que, embora os três sinais possuam a mesma frequência fundamental, seus espectros são distintos devido ao formato de cada onda no domínio do tempo. O sinal senoidal apresenta energia concentrada apenas na frequência fundamental (em 5 Hz), enquanto a onda quadrada exibe múltiplos harmônicos ímpares, evidenciando sua descontinuidade temporal. Já o sinal “dente de serra” (triangular) possui harmônicos pares e ímpares, com amplitudes decrescentes, refletindo sua variação linear abrupta. Assim, o formato do sinal no tempo influencia diretamente sua distribuição de energia no domínio da frequência.

O código utilizado para gerar os gráficos, apresentado a seguir, cria três sinais periódicos distintos (senoidal, onda quadrada e “dente de serra”) com a mesma

frequência e taxa de amostragem, utilizando as bibliotecas NumPy e SciPy. Em seguida, aplica-se a transformada tempo–frequência por meio do espectrograma, que divide o sinal em janelas sobrepostas, calcula a Transformada Rápida de Fourier (FFT) de cada segmento e exibe a distribuição de energia ao longo do tempo e da frequência. Por fim, os resultados são plotados com o Matplotlib em subgráficos organizados, permitindo a comparação visual entre os diferentes sinais e evidenciando como o formato no domínio do tempo afeta o conteúdo espectral.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import square, sawtooth, spectrogram

# Parâmetros
fs = 1000          # taxa de amostragem
duracao = 10       # 10 segundos
t = np.linspace(0, duracao, fs * duracao)
A = 2
freq = 5

# Sinais
sinal1 = A * np.sin(2 * np.pi * freq * t)
sinal2 = A * square(2 * np.pi * freq * t)
sinal3 = A * sawtooth(2 * np.pi * freq * t)

def plot_spectrogram(sinal, ax, titulo):
    f, ts, Sxx = spectrogram(
        sinal,
        fs,
        window='hann',
        nperseg=2048,
        noverlap=1800,
        nfft=4096
    )
    ax.pcolormesh(ts, f, 10*np.log10(Sxx),
        shading='gouraud')
    ax.set_ylim(0, 50)
    ax.set_ylabel('Frequência (Hz)')
    ax.set_title(titulo)

fig, axs = plt.subplots(3, 1, figsize=(8, 8))

plot_spectrogram(sinal1, axs[0], '(a) Sinal senoidal')
plot_spectrogram(sinal2, axs[1], '(b) Onda quadrada')
plot_spectrogram(sinal3, axs[2], '(c) Onda dente de
serra')
```

```

    axs[2].set_xlabel('Tempo (s)')
    plt.tight_layout()
    plt.show()

```

Código-fonte 4 - Plotando espectrograma de sinais periódicos

Fonte: Elaborado pelo autor (2025)

1.1.3 Sinais analógicos e sinais digitais

Sinais analógicos e sinais digitais são formas distintas de representar informações em sistemas eletrônicos e de comunicação. O sinal analógico varia de maneira contínua no tempo e na amplitude, podendo assumir infinitos valores dentro de um intervalo, o que permite representar fenômenos físicos naturais, como som e temperatura, com alta fidelidade. Já o sinal digital é discreto, assumindo valores bem definidos, geralmente binários (0 e 1), obtidos a partir da amostragem e quantização de um sinal analógico, o que facilita o processamento, a compressão, a correção de erros e a reprodução fiel da informação, mesmo na presença de interferências.

A amostragem é o processo pelo qual um sinal analógico contínuo no tempo é convertido em um sinal discreto, por meio da medição de sua amplitude em instantes de tempo igualmente espaçados. Cada uma dessas medições, chamadas de amostras, representa o valor do sinal naquele instante específico e permite que a informação seja armazenada, processada e transmitida em sistemas digitais. A qualidade da representação do sinal original depende diretamente da taxa de amostragem, isto é, do número de amostras coletadas por segundo.

O Teorema de Nyquist–Shannon estabelece que, para que um sinal analógico possa ser reconstruído fielmente a partir de suas amostras sem perda de informação, a frequência de amostragem deve ser, no mínimo, o dobro da maior frequência presente no sinal. Matematicamente, se f_{max} é a frequência máxima do sinal, a taxa de amostragem f_s deve satisfazer:

$$f_s \geq 2f_{max}$$

Esse limite, conhecido como frequência de Nyquist, evita o fenômeno do aliasing, no qual componentes de alta frequência são interpretadas erroneamente como frequências mais baixas no sinal digital. Como comparação, você pode ver o aliasing acontecendo devido a taxa de amostragem do olho humano: se uma hélice girar muito rapidamente, ela pode parecer estar girando mais devagar, parada ou até

mesmo no sentido contrário, porque o sistema visual não consegue “amostrar” o movimento com frequência suficiente para representar corretamente a rotação real.

Um sinal pode ser representado como uma lista em Python, na qual cada elemento corresponde a uma amostra do sinal em um instante específico de tempo. Essa lista contém valores numéricos que representam a amplitude do sinal após o processo de amostragem.

A relação com a frequência de amostragem f_s é direta: f_s define quantas amostras são coletadas por segundo, determinando o espaçamento temporal entre os elementos da lista, dado por $T_s = \frac{1}{f_s}$. Assim, quanto maior for f_s , maior será o número de elementos na lista para um mesmo intervalo de tempo e mais fiel será a representação do sinal original; por outro lado, uma frequência de amostragem baixa reduz o número de amostras e pode causar perda de informação, especialmente em sinais com componentes de alta frequência.

Observe o código-fonte “Plotando espectrograma de sinais periódicos” como exemplo: a taxa de amostragem foi definida como 1000 amostras por segundo e a duração do sinal definida como 10 segundos; logo, o tamanho da lista em Python para a variável `signal1` (assim como para as variáveis `signal2`, `signal3` e `t` de tempo) é de $1000 \times 10 = 10000$ elementos.

A quantização, por sua vez, é a etapa do processo de digitalização em que as amplitudes contínuas de um sinal amostrado são aproximadas por um conjunto finito de valores discretos. Após a amostragem definir quando o sinal é medido, a quantização define quais valores essas medições podem assumir, de acordo com a resolução do sistema, normalmente expressa em número de bits. Cada amostra é então associada ao nível discreto mais próximo, o que introduz um erro inevitável chamado erro de quantização. Quanto maior o número de níveis (ou bits), menor é esse erro e mais fiel é a representação do sinal original, resultando em maior qualidade e menor distorção no sinal digital.

Em um computador ou celular moderno, os sinais de áudio são processados por meio de uma cadeia de etapas que converte sinais físicos analógicos em sinais digitais e vice-versa. Inicialmente, o som captado pelo microfone é transformado em um sinal elétrico analógico, que passa por um conversor analógico-digital (ADC), responsável pela amostragem, quantização e codificação do sinal em formato digital.

Esses dados digitais são então processados pelo sistema, podendo sofrer filtragem, compressão, mixagem e aplicação de efeitos por meio de algoritmos de processamento digital de sinais (DSP). Para a reprodução, o processo inverso ocorre: o áudio digital é convertido em sinal analógico por um conversor digital-analógico (DAC), amplificado e enviado aos alto-falantes ou fones de ouvido.

1.1.4 Arquivos de áudio

Para o sistema computacional, o áudio amostrado é representado na memória de trabalho como uma lista de elementos. Na memória permanente, a lista precisa ser armazenada como arquivo. Existem vários tipos de arquivos utilizados para armazenar áudio, cada um com uma especificidade. De modo geral, esses arquivos podem ser divididos em duas grandes categorias: arquivos que armazenam o áudio propriamente dito e arquivos que armazenam apenas instruções musicais.

Os arquivos de áudio digital guardam o som na forma de amostras, ou seja, valores numéricos que representam a variação do sinal sonoro ao longo do tempo. Esses arquivos podem ser sem compressão ou com compressão. Nos formatos sem compressão, como o WAV, todas as amostras do áudio são armazenadas integralmente, preservando a qualidade original do som. A principal desvantagem desse tipo de arquivo é o grande tamanho ocupado na memória permanente.

Já os formatos de áudio com compressão buscam reduzir o tamanho do arquivo. A compressão pode ser sem perda de informação, como no formato FLAC, em que o áudio original é totalmente recuperado durante a reprodução, ou com perda de informação, como nos formatos MP3, AAC e OGG. Nesse último caso, partes do sinal sonoro consideradas menos perceptíveis ao ouvido humano são descartadas, o que diminui significativamente o tamanho do arquivo, porém com possível redução da qualidade sonora.

Além dos arquivos que armazenam o áudio amostrado, existem os arquivos de instruções musicais, sendo o MIDI o principal exemplo. Diferentemente dos arquivos de áudio digital, o MIDI não armazena o som em si, mas sim comandos que descrevem a música, como notas, duração, intensidade, tempo e o instrumento a ser utilizado. Durante a reprodução, essas instruções são interpretadas por um sintetizador ou instrumento virtual, que gera o som correspondente. Por essa razão,

MIDI files have a very small size and can sound differently depending on the device or software used for execution.

Just as MIDI, MusicXML does not store audio properly, but musical information. However, its main focus is the representation of musical notation, or in other words, the way music is written in a score. This format describes elements such as notes, measures, clefs, staves, rhythmic figures and articulations, allowing the exchange of scores between different music editing software. Unlike MIDI, which prioritizes execution and sound reproduction, MusicXML is more used for editing, visualization and sharing of scores. For this reason, it is not intended for the direct reproduction of sound, but rather to ensure fidelity in musical writing, being able to be later interpreted by programs that generate audio from this information.

In the table below, some types of files used to store or synthesize audio on the computer are summarized:

Tipo de Arquivo	Armazena Áudio	Compressão	Perda de Qualidade	Característica Principal
WAV	Sim	Não	Não	Alta qualidade, tamanho grande.
FLAC	Sim	Sim	Não	Qualidade original.
MP3	Sim	Sim	Sim	Compacto, proprietário.
OGG	Sim	Sim	Sim	Compacto, aberto.
MIDI	Não	Não se aplica	Não se aplica	Instruções musicais.
MusicXML	Não	Não se aplica	Não se aplica	Partitura digital.

Quadro 1 - Alguns tipos de arquivos de áudio
Fonte: Elaborado pelo autor (2025)

1.1.5 Audacity

Audacity is a free and open-source audio editing and processing software, in addition to being multiplatform (Windows, macOS and Linux). It allows recording, importing and exporting different audio formats and applying effects such as cutting, normalization, amplification, noise reduction, equalization and speed or pitch change. Audacity is especially useful for demonstrating basic concepts in a

prática, como edição no domínio do tempo, manipulação de ondas sonoras e aplicação de filtros, ajudando os alunos a visualizar e ouvir diretamente os resultados das transformações feitas no sinal de áudio. Faça o download do Audacity em <<https://www.audacityteam.org/>> para acompanhar os próximos exemplos.

Na figura a seguir, é apresentada a interface de abertura do Audacity:

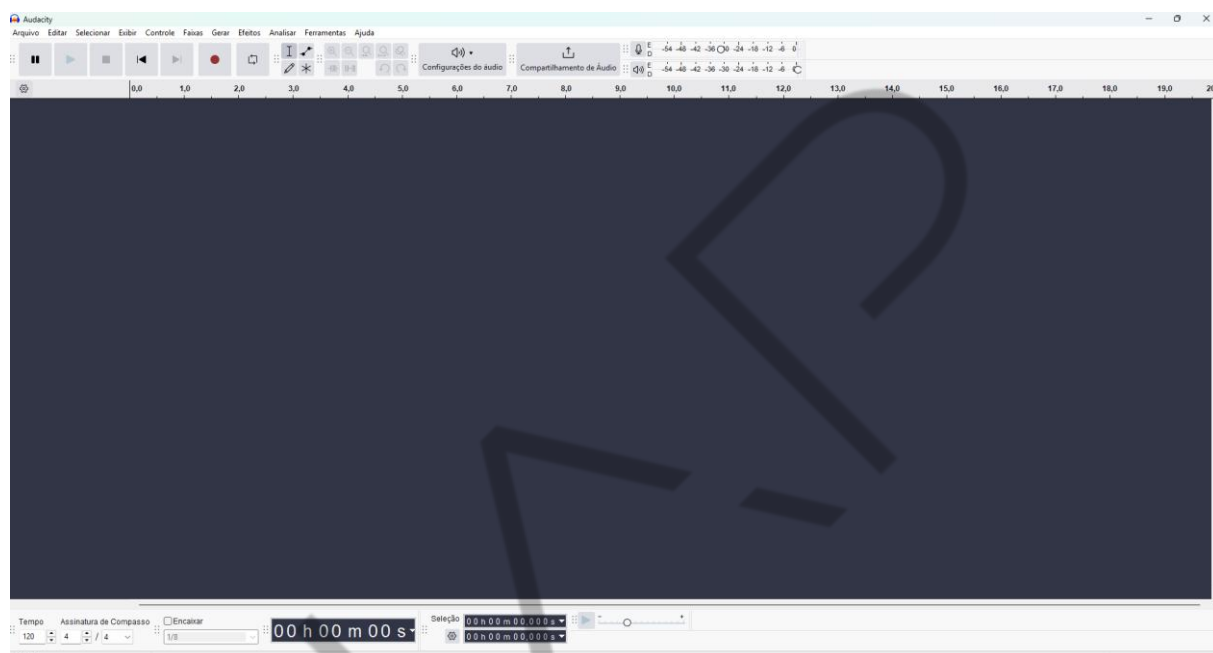


Figura 5 - Interface do Audacity
Fonte: Elaborado pelo autor (2025)

Para gravar um som, basta ter um microfone no computador e clicar no botão de círculo vermelho. Para finalizar a gravação, clique no botão quadrado preto. Por exemplo, gravamos um áudio como mostrado a seguir:

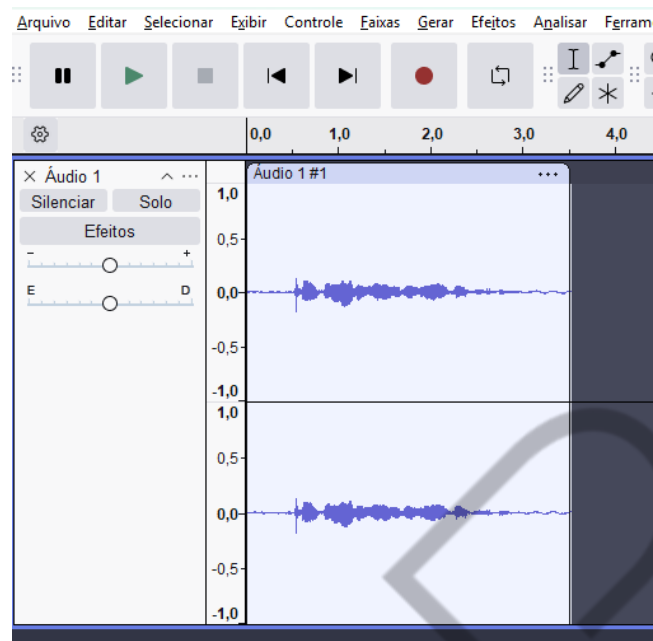


Figura 6 - Gravando um áudio no Audacity
Fonte: Elaborado pelo autor (2025)

Na figura anterior, vemos duas faixas com o sinal de áudio gravado. Chamamos isso de canais ou faixas de áudio. No caso, estamos com um áudio estéreo, já que gravamos o áudio em dois canais de maneira que o sinal está repetido em cada um deles. Podemos dar zoom (selecionando uma região do áudio e girando o scroll do mouse) para identificar a forma de onda periódica no domínio do tempo, como mostrado na figura a seguir:

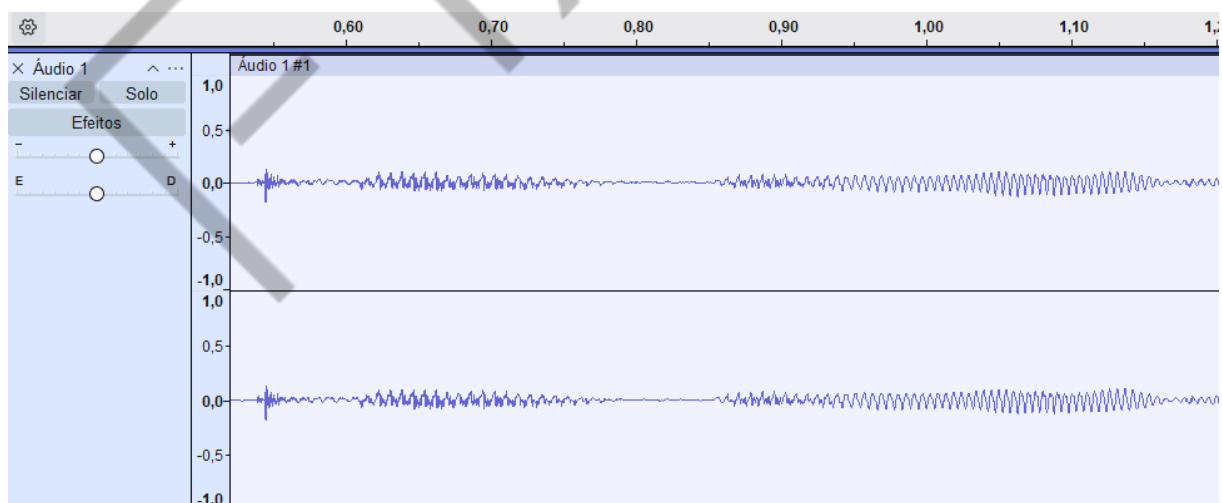


Figura 7 - Zoom no domínio do tempo
Fonte: Elaborado pelo autor (2025)

Também é possível ver o mesmo áudio no domínio da frequência (espectrograma), como fizemos em Python, selecionando essa opção nos “três pontinhos” acima de “Solo”:

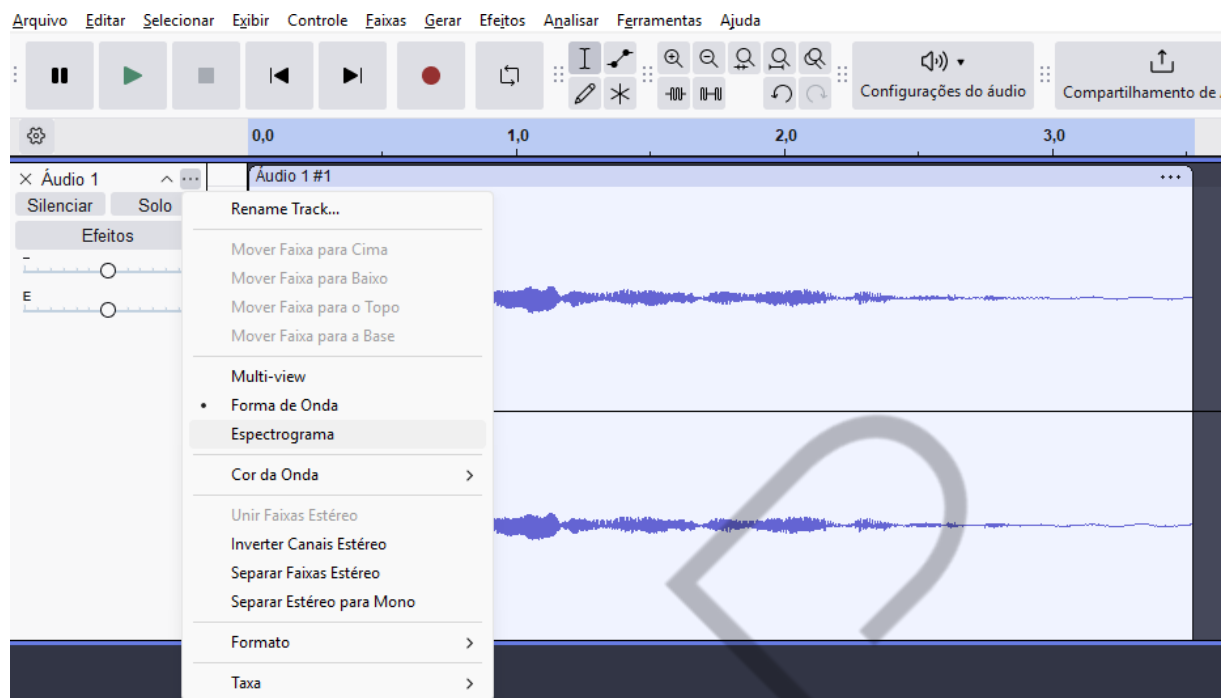


Figura 8 - Alterando a exibição do áudio no Audacity para espectrograma
Fonte: Elaborado pelo autor (2025)

O resultado é apresentado na próxima figura, em que podemos ver as componentes de frequência. A fala gravada é a parte mais intensa com ondulações de frequência (as repetições das ondulações subindo em frequência são as harmônicas); o tracejado em 6000 Hz é um ruído eletrônico intermitente (pulsado).

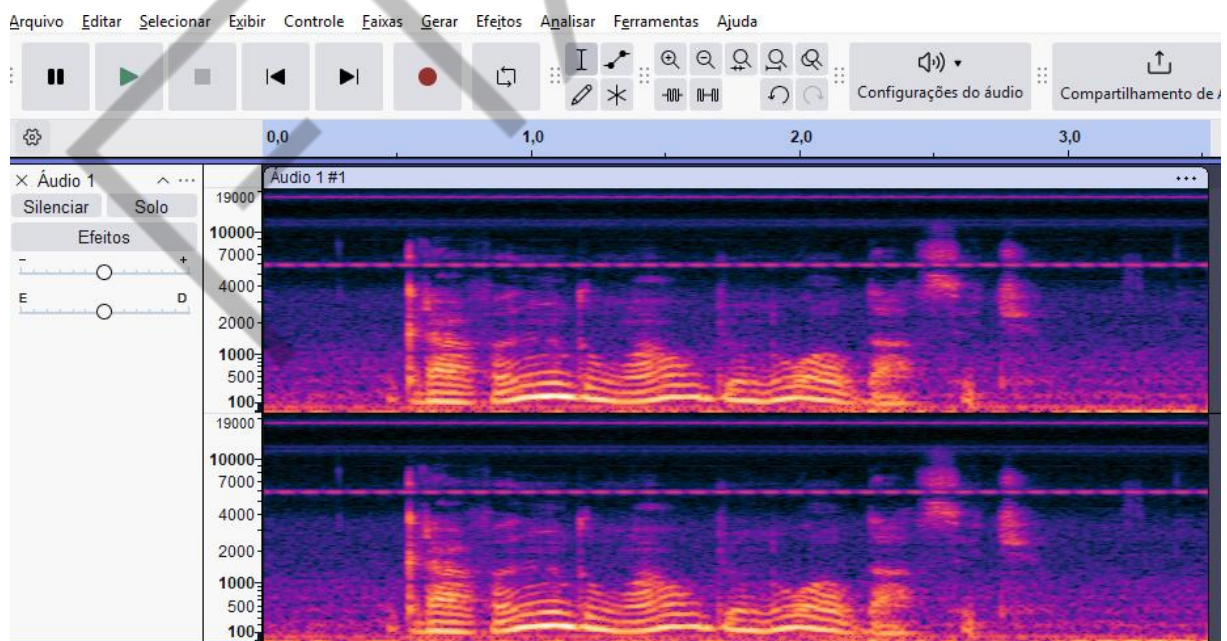


Figura 9 - Espectrograma do áudio no Audacity
Fonte: Elaborado pelo autor (2025)

A vantagem de se trabalhar no domínio da frequência é que podemos observar algumas etapas de filtragem. Vamos aplicar um efeito de filtro de Grave e Agudos.

Lembre-se: “graves” são frequência mais baixas, enquanto “agudos” são frequências mais altas. Para selecionar o efeito, siga o procedimento mostrado na próxima figura:

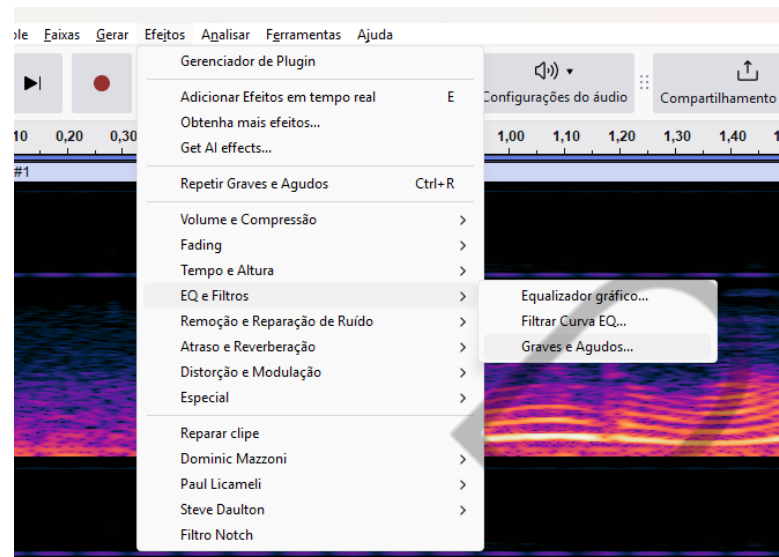


Figura 10 - Aplicando filtro de Graves e Agudos
Fonte: Elaborado pelo autor (2025)

Na janela do filtro, aplique – 30dB nos agudos, conforme mostrado a seguir:

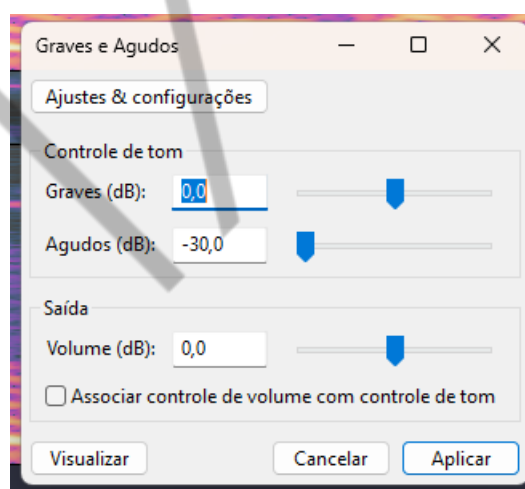


Figura 11 - Removendo agudos no Audacity
Fonte: Elaborado pelo autor (2025)

Depois de clicar em aplicar, observe que as componentes de frequência alta ficaram escuras, indicando que são menos intensas (possuem menos energia), enquanto as componentes de frequência mais baixa mantêm suas intensidades (figura “Espectrograma do áudio após filtragem de agudos”).

Observação: foi dado um zoom no sinal para facilitar visualização clicando no ícone da lupa “ajustar projeto à largura”.

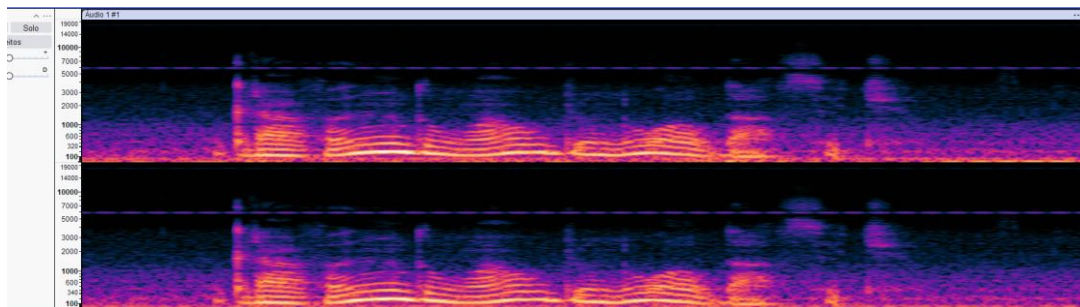


Figura 12 - Espectrograma do áudio após filtragem de agudos
Fonte: Elaborado pelo autor (2025)

Também podemos editar o áudio diretamente na interface, removendo as partes com pouco áudio no começo e no fim da gravação. É possível reproduzir o áudio no botão verde de play ou apertando a barra de espaço para verificar quais partes do seu áudio não possuem som de interesse (como fala). Com o mouse, selecionamos uma parte do áudio e, em seguida, pressionamos o botão delete para removê-la, conforme mostrado na figura “Espectrograma do áudio antes e depois da remoção de 0,5s iniciais”.

Para exportar o áudio após a edição, basta clicar em “Arquivo” e depois em “Exportar Áudio como”. Uma janela com opções irá aparecer. Observe na figura “Exportando arquivo de áudio no Audacity” algumas questões que discutimos agora: você pode exportar o áudio com dois canais (estéreo) ou um único canal (mono); a taxa de amostragem é de 44100 Hz para estar pelo menos duas vezes acima da maior frequência que o ouvido humano captura; a qualidade é referente a quantidade de bits por segundo utilizados no padrão de compressão MP3:

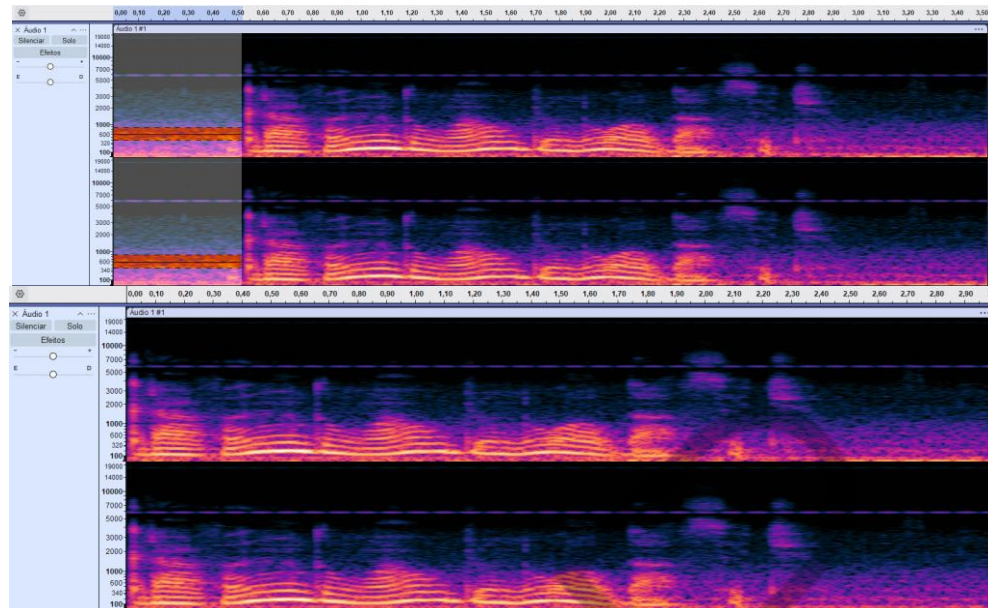


Figura 13 - Espectrograma do áudio antes e depois da remoção de 0,5s iniciais
Fonte: Elaborado pelo autor (2025)

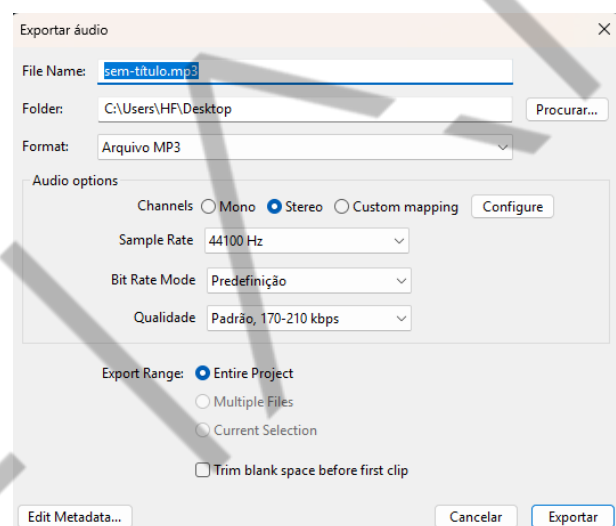


Figura 14 - Exportando arquivo de áudio no Audacity
Fonte: Elaborado pelo autor (2025)

Existem muitas outras funcionalidades que podemos explorar no Audacity para editar áudio, incluindo uma vasta gama de efeitos e a possibilidade de mixagem de canais (ter novos canais com gravações diferentes). Contudo, na próxima seção, iremos tratar de como o Aprendizado de Máquina (ML) pode ser utilizado no Processamento de Sinais de Áudio.

2 MACHINE LEARNING IN AUDIO PROCESSING

2.1 Using machine learning in audio processing

In audio signal processing, machine learning is used as a mechanism to learn statistical relationships between the signal and a desired task, operating in conjunction with techniques that represent, organize or transform this signal. The starting point is always a continuous acoustic signal, converted into a digital signal by sampling and quantization, resulting in a sequence of values that represent the variation of sound pressure over time. This raw signal can be processed directly by machine learning models or can pass through intermediate transformations that reorganize its information.

In approaches based on feature extraction (also known as feature extraction), the processing of signals is used explicitly to convert the signal in time into more structured representations. A transformation very commonly used is the Short-Time Fourier Transform (STFT), which divides the signal into short temporal windows and calculates, in each one, the content in frequencies, producing a time-frequency representation called **spectrogram**. This representation makes explicit the spectral evolution of audio, which is associated with perceptual properties such as timbre (the characteristic that allows us to distinguish sounds with the same pitch and intensity, but produced by different sources) and pitch (also called *pitch* and related to whether a sound is low or high pitched).

From the spectrogram, more compact descriptors can be calculated, such as Mel-frequency cepstral coefficients (MFCCs), which approximate the way the human ear distributes resolution in frequency. These feature vectors are then used as input for machine learning models, which learn to associate patterns in these vectors with labels or values of interest, such as phonemes, words, types of sound or acoustic states.

In approaches called end-to-end, common in deep learning, the explicit separation between signal processing and machine learning is reduced. The model receives as input the signal itself in the time domain or a representation very close to it, such as the basic spectrogram, and learns internally the necessary transformations for the task. Initial layers of the network

neural passam a desempenhar um papel análogo a filtros de processamento de sinais, aprendendo automaticamente operações semelhantes a bancos de filtros, detecção de energia em bandas de frequência e combinações temporais, sem que essas operações sejam pré-definidas. À medida que os dados avançam pelas camadas, o modelo constrói representações progressivamente mais abstratas, capturando regularidades temporais, harmônicas e rítmicas relevantes.

Em ambos os casos, o processamento de sinais fornece a estrutura matemática do problema (domínio do tempo, domínio da frequência, resolução temporal e espectral), enquanto o aprendizado de máquina fornece os mecanismos de estimação e generalização, ajustando parâmetros a partir de dados para lidar com variabilidade, ruído e ambiguidade inerentes aos sinais de áudio.

Na figura a seguir, é apresentado um diagrama esquemático das diferentes abordagens. Neste diagrama, explicitamos o pré-processamento de filtragem que pode envolver redução de ruído, equalização de canal e filtragem de frequências: em (a) temos um pipeline completo pela abordagem de extração de features; em (b) um modelo de ML é utilizado diretamente sobre o sinal pré-filtrado para gerar a saída esperada (um exemplo de end-to-end com filtros clássicos); em (c) o modelo de ML é end-to-end, incluindo capacidades de filtros internamente ao modelo.

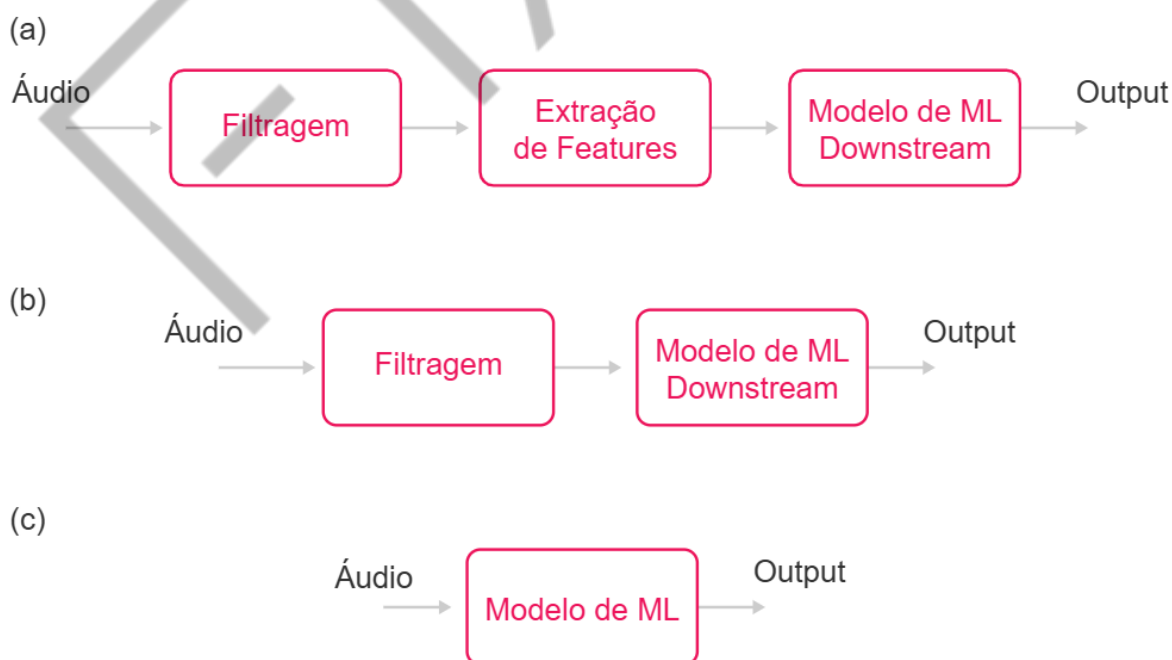


Figura 15 - Abordagens de uso de ML no processamento de áudio
Fonte: Elaborado pelo autor (2025)

Por downstream nos referimos a uma tarefa final realizada sobre o áudio pré-processado. A integração entre processamento de sinais de áudio e aprendizado de máquina permite formular diversas tarefas como problemas de estimação ou decisão a partir de sinais acústicos (tarefas downstream). Entre as tarefas mais comuns está a classificação de sons, na qual um trecho de áudio é associado a um rótulo discreto, como eventos acústicos, cenas sonoras ou instrumentos musicais. O modelo aprende padrões espectrais e temporais característicos de cada classe, seja a partir de características extraídas ou diretamente do sinal, com o objetivo de atribuir um rótulo discreto a um trecho de áudio, seja para identificar tipos de eventos acústicos (passos, buzinas, chuva), cenas sonoras (rua, escritório, floresta) ou instrumentos musicais.

Ainda na classificação, o reconhecimento automático de fala busca mapear um sinal contínuo para unidades linguísticas, enquanto o reconhecimento de voz foca na identificação ou verificação de falantes, explorando características relativamente estáveis da produção vocal. Há ainda tarefas como detecção e segmentação de eventos sonoros, separação e realce de fontes, detecção de fala, análise de emoção e indexação e recuperação de áudio, todas baseadas no aprendizado de padrões acústicos relevantes para cada objetivo.

No contexto de modelos generativos aplicados a áudio, o aprendizado de máquina é utilizado para modelar a distribuição estatística dos sinais sonoros e gerar novas amostras consistentes. Na síntese de fala, o modelo converte representações linguísticas em áudio, aprendendo simultaneamente aspectos como prosódia e dinâmica temporal. De forma relacionada, técnicas de clonagem e adaptação de voz permitem condicionar a geração de fala à identidade vocal de um indivíduo, separando conteúdo linguístico e características do falante. A geração de música envolve a modelagem de estruturas rítmicas, harmônicas e tímbricas ao longo do tempo, podendo operar diretamente sobre o sinal de áudio ou sobre representações intermediárias. Outras aplicações incluem continuação e completamento de áudio, transferência de estilo e geração condicionada por controles acústicos ou musicais.

Em todas essas abordagens, o processamento de sinais fornece os fundamentos para representar, transformar e reconstruir o áudio, enquanto os modelos de aprendizado de máquina aprendem como combinar essas representações para análise ou síntese de sinais sonoros. Os detalhes de implementação e arquiteturas específicas serão explorados a seguir.

2.2 Extração de features

Uma forma conveniente de usar aprendizado de máquina para processar sinais de áudio é utilizando um pré-processamento de extração de features. Nessa etapa, o sinal de áudio é processado para que certas características sejam extraídas dele. Há vários tipos de estratégias e técnicas para isso comumente adotadas na área de processamento de sinais. Entre as mais utilizadas estão as características no **domínio do tempo, da frequência e do tempo-frequência**.

No domínio do tempo, extraem-se métricas simples como energia, taxa de cruzamento por zero e amplitude média, que fornecem informações básicas sobre a dinâmica do sinal. No domínio da frequência, obtido via Transformada de Fourier, são extraídas features como espectro de magnitude, centroide espectral, largura de banda e roll-off, que descrevem a distribuição de energia nas frequências.

Já no domínio tempo-frequência, técnicas como a Transformada de Fourier de Tempo Curto permitem analisar variações espectrais ao longo do tempo. A partir dela, derivam-se representações amplamente utilizadas, como espectrogramas e coeficientes cepstrais em escala Mel (MFCCs), que são especialmente eficazes por aproximarem a percepção auditiva humana.

Essas features reduzem a dimensionalidade do problema e tornam os dados mais informativos para os algoritmos de aprendizado de máquina, facilitando tarefas como classificação, detecção e reconhecimento de padrões em sinais de áudio.

Vamos fazer um exemplo de extração de features de um áudio simples. Usando o Audacity, grave um áudio falando uma frase, como “extraíndo features com Python”. Exporte como MP3, mono com taxa de amostragem padrão. Coloque o arquivo gerado na mesma pasta do código em Python. Extrairemos as seguintes features:

- **Centroide espectral:** indica onde está o “centro de massa” do espectro. Sons mais agudos implicam centroide maior, enquanto sons mais graves implicam centroide menor.
- **Largura de banda espectral:** mede a dispersão das frequências em torno do centroide. Sons mais “espalhados” no espectro possuem largura maior.

- **Taxa de cruzamento por zero (ZCR):** conta quantas vezes o sinal cruza o valor zero. Sons ruidosos ou agudos possuem ZCR maior, enquanto sons suaves ou graves tendem a ter uma ZCR menor.
- **Coeficientes Cepstrais em Escala Mel (MFCC):** Representam características perceptuais do som, sendo muito usados em Reconhecimento de Fala, Reconhecimento de Voz e Classificação de Música.
- Para realizar o processamento de sinais necessário para a extração de features, vamos utilizar uma biblioteca pronta em Python chamada **librosa**. Também instalaremos o **soundfile** para salvar arquivos de áudio. Basta executar o seguinte prompt no terminal:

```
pip install librosa soundfile
```

Comando de prompt 1 - Instalação do Librosa
Fonte: Elaborado pelo autor (2025)

No código-fonte a seguir, temos os procedimentos de extração:

```
import librosa
import numpy as np

audio, sr = librosa.load("audio1.mp3", sr=None)

print("Quantidade de amostras:", len(audio))
print("Duração em segundos:", len(audio)/sr)

# Features
mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=5)
spectral_centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)
spectral_bandwidth = librosa.feature.spectral_bandwidth(y=audio, sr=sr)
zero_crossing_rate = librosa.feature.zero_crossing_rate(audio)

print("Número de frames aproximado:", len(audio)/512)
print("Número de frames real:", len(spectral_centroid[0]), len(spectral_bandwidth[0]), len(zero_crossing_rate[0]), len(spectral_centroid[0]))

# Agregação estatística (média dos frames)
features = np.hstack([
    mfccs.mean(axis=1),
```

```
spectral_centroid.mean(axis=1),
spectral_bandwidth.mean(axis=1),
zero_crossing_rate.mean(axis=1)
])

print("Vetor de features:", features)
print("Dimensão do vetor:", features.shape)
```

Código-fonte 5 - Extração de features de um sinal de áudio

Fonte: Elaborado pelo autor (2025)

Vamos entender o código. Inicialmente, carregamos o arquivo de áudio utilizando o comando `load`: na variável `áudio`, teremos a lista com os valores de amplitude do sinal ao longo do tempo, e na variável `sr`, a taxa de amostragem usada. Observe o resultado dos dois prints executados na sequência: teremos o tamanho da lista, ou seja, a quantidade de amostras que forma o áudio e a duração dele, calculada usando a relação de que a taxa de amostragem é exatamente a quantidade de amostras digitalizadas em um segundo de sinal.

Na sequência, são calculadas as quatro features que queríamos. Entretanto, aqui há um detalhe importante: cada feature será calculada ao longo do tempo, através de frames. Um frame é uma janela de número fixo de amostras sobre o áudio. No caso, vamos utilizar o padrão do librosa, no qual cada frame é composto de 2048 amostras com um deslocamento entre frames (`hop_length`) de 512 amostras. Isso significa que um novo frame começa a cada 512 amostras, tendo, portanto, sobreposição entre os frames. Printamos o número aproximado de frames, dado pelo total de amostras (tamanho do sinal) dividido pelo `hop_length`, e o número real de frames, dado pelo tamanho do vetor da feature extraída (printamos quatro vezes para mostrar que as quatro features foram calculadas com as mesmas janelas). Veja na figura a seguir um resultado esperado para esse código:

```
Quantidade de amostras: 158107
Duração em segundos: 3.585192743764172
Número de frames aproximado: 308.802734375
Número de frames real: 309 309 309 309
Vetor de features: [-4.37270020e+02  1.31813004e+02  2.14502754e+01  4.02263222e+01
 1.43556395e+01  3.04825292e+03  4.10368338e+03  7.91916338e-02]
Dimensão do vetor: (8,)
```

Figura 16 - Features extraídas, saída do Código-fonte 5

Fonte: Elaborado pelo autor (2025)

Nosso sinal tem 158107 amostras; utilizamos 2048 amostras para calcular uma feature (por exemplo, contar quantas vezes o sinal passa por zero); com 2048 não temos o sinal todo, precisamos deslocar a janela para frente e calcular de novo a

feature para uma posição mais adiantada do áudio; em vez de fazermos isso para janelas individuais, fazemos para janelas sobrepostas, de forma que uma parte do sinal já calculado será usado para calcular a feature na próxima janela; assim, temos no final 309 valores da contagem de quantas vezes o sinal passa por zero, sendo cada um desses valores respectivo a uma janela do sinal. A pergunta que falta responder é, qual desses 309 valores usar como feature final para o modelo de aprendizado de máquina?

Em geral, uma abordagem possível é calcular uma estatística sobre os frames. A mais utilizada é a média. Ou seja, estamos calculando a taxa média de cruzamento por zero ao longo de todo o tempo de duração do sinal. O mesmo vale para as outras features. Contudo, devemos chamar atenção que os MFCC são vários coeficientes de forma que, na verdade, essa feature corresponde em si mesma a cinco features (número de coeficientes que estamos extraindo do sinal). Dessa forma, o vetor de features final corresponde a 8 valores (5 coeficientes MFCC e as outras 3 features que escolhemos) que, na verdade, são a média dessas características para as 309 janelas usadas. Ufa! Temos agora uma assinatura desse arquivo de áudio representada por esses 8 valores. Podemos utilizar isso para uma downstream task, como classificação de áudio. Entretanto, observe que precisaríamos de mais exemplos de áudio, aqui só tratamos 1 áudio. Para fazer aprendizado de máquina, precisaríamos começar com pelo menos uns 50 áudios!

Agora que você já tem uma ideia clara de quais são os passos necessários para gerar exemplos e extrair features deles, vamos aplicar tudo por meio de um exemplo de reconhecimento de voz com dados já pré-processados.

2.3 Classificação de áudio por features

Para treinar um modelo de aprendizado de máquina para realizar a classificação de áudio, precisamos um dataset contendo vários exemplos de áudio. No caso, iremos usar apenas as impressões digitais desses áudios, isto é, o vetor de features que o caracteriza. No repositório público OpenML, há um dataset com 990 exemplos de áudio utilizado para reconhecimento de vogais. No código-fonte a seguir, fazemos o download deste dataset:

```
from sklearn.datasets import fetch_openml

# 1. Baixando dados do OpenML
dataset = fetch_openml(data_id=307, as_frame=True).frame
dataset.head()
```

Código-fonte 6 - Baixando dataset Vowel do OpenML
Fonte: Elaborado pelo autor (2025)

Na próxima figura, mostramos o cabeçalho deste dataset:

	Speaker_Number	Sex	Feature_0	Feature_1	Feature_2	Feature_3	Feature_4	Feature_5	Feature_6	Feature_7	Feature_8	Feature_9	Class
0	Andrew	Male	-3.639	0.418	-0.670	1.779	-0.168	1.627	-0.388	0.529	-0.874	-0.814	hId
1	Andrew	Male	-3.327	0.496	-0.694	1.365	-0.265	1.933	-0.363	0.510	-0.621	-0.488	hId
2	Andrew	Male	-2.120	0.894	-1.576	0.147	-0.707	1.559	-0.579	0.676	-0.809	-0.049	hEd
3	Andrew	Male	-2.287	1.809	-1.498	1.012	-1.053	1.060	-0.567	0.235	-0.091	-0.795	hAd
4	Andrew	Male	-2.598	1.938	-0.846	1.062	-1.633	0.764	0.394	-0.150	0.277	-0.396	hYd

Figura 17 - Dataset Vowel, saída do Código-fonte 6
Fonte: Elaborado pelo autor (2025)

Observe que, para cada áudio, foram extraídas 10 features, cuja natureza exata não nos é informada. Para cada um deles, temos o nome do falante, o sexo do falante e a vogal falada. Neste exemplo, construiremos um reconhecedor de voz, ou seja, nosso objetivo será usar as 10 features para classificar o falante (Speaker_Number).

Primeiramente, precisamos confirmar se temos dados para isso. Ao executar `dataset.groupby('Speaker_Number').count()`, verificamos que há 15 falantes diferentes, cada um com 66 áudios diferentes. Logo, já dispomos de dados suficientes para prosseguir para a downstream task de interesse.

No código-fonte a seguir, damos continuidade a criação do reconhecedor de voz, separando as colunas (features/target) e linhas (treino/teste) do dataset. Em seguida, utilizamos um algoritmo de Random Forest para treinar um classificador com os dados. O classificador treinado é então usado sobre o conjunto de teste para que possamos verificar seu desempenho.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report,
confusion_matrix

# 2. Separar features e target
X = dataset.drop(columns=["Speaker_Number", "Sex",
"Class"])
y = dataset["Speaker_Number"]

# 3. Separar treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# 4. Treinar classificador
clf = RandomForestClassifier(n_estimators=100,
random_state=42)
clf.fit(X_train, y_train)

# 5. Avaliação do Desempenho
y_pred = clf.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm,annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de Confusão")
plt.xlabel("Predito")
plt.ylabel("Real")
plt.show()

print(classification_report(y_test,y_pred))
```

Código-fonte 7 - Treinando um reconhecedor de voz com o dataset Vowel
Fonte: Elaborado pelo autor (2025)

O resultado da avaliação de desempenho do modelo é apresentado na próxima figura. Observe que as features utilizadas e a quantidade de dados foi suficiente para permitir treinar o modelo para reconhecer a voz de cada um dos 15 falantes. A acurácia alcançada é de 91%, com alguns falantes atingindo 100%. Verificando os resultados, você é capaz de dizer quais foram os falantes que o modelo teve a maior dificuldade de reconhecer a voz?

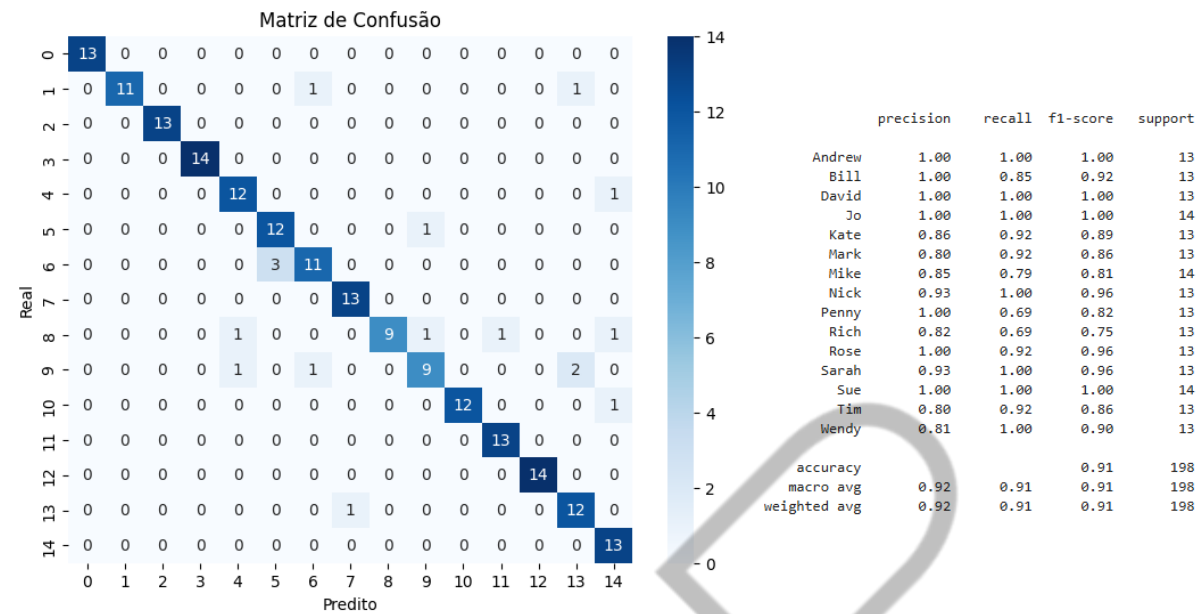


Figura 18 - Resultados do reconhecedor de voz, saída do Código-fonte 7
Fonte: Elaborado pelo autor (2025)

IMPORTANTE: Reconhecimento de Voz pode se referir a Reconhecimento de Falante (Speaker recognition) ou Reconhecimento de Fala (Speech recognition). No exemplo tratado aqui, e na maior parte deste texto, utilizamos Reconhecimento de Voz como sinônimo de Reconhecimento de Falante.

2.4 Remoção de ruído de áudio por sistema end-to-end

Neste exemplo, iremos aplicar uma rede neural para realizar a etapa de remoção de ruído de um áudio. Primeiro, vamos construir um áudio de trabalho. Existem vários sites com músicas e efeitos sonoros gratuitos com licença livre, por exemplo: <https://soundbible.com/>, <https://pixabay.com/> e <https://freemusicarchive.org/>. Iremos baixar uma música e um barulho de sirene desses sites (no exemplo, baixamos <https://soundbible.com/581-Police-Siren-3.html> e <https://pixabay.com/music/funk-running-night-393139/>).

Com o Audacity, podemos combinar e cortar esses sons. Abra o Audacity, solte e arraste os dois arquivos baixados para trilha. O resultado deve ser:

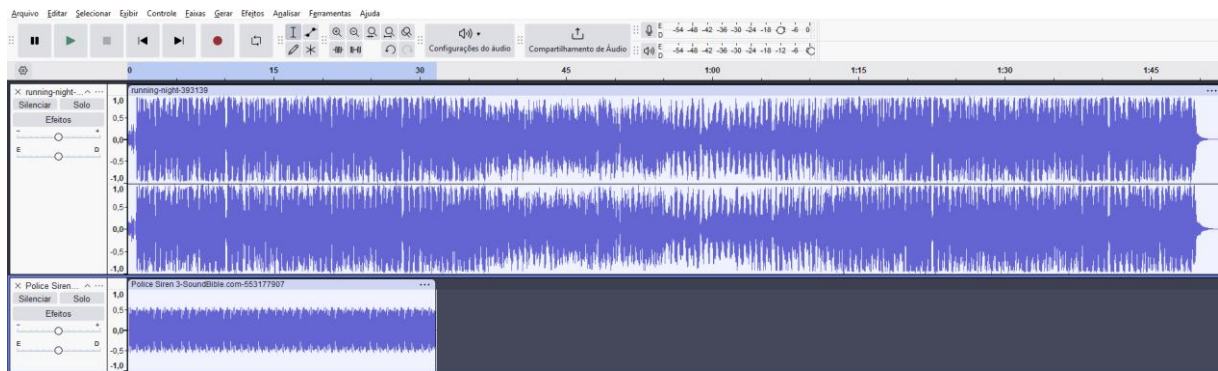


Figura 19 - Combinando áudios com o Audacity
Fonte: Elaborado pelo autor (2025)

Vamos cortar o início da música e depois o final para que ela fique com o mesmo tamanho do ruído. Para fazer isso, selecione com o mouse e depois clique em delete. O resultado será:

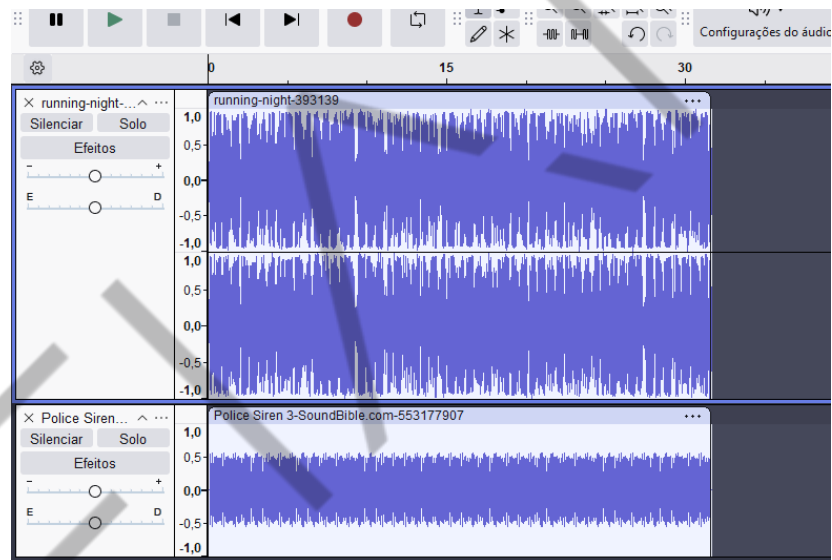


Figura 20 - Ajustando a duração de áudios combinados com o Audacity
Fonte: Elaborado pelo autor (2025)

Observe os espectrogramas dos mesmos áudios:

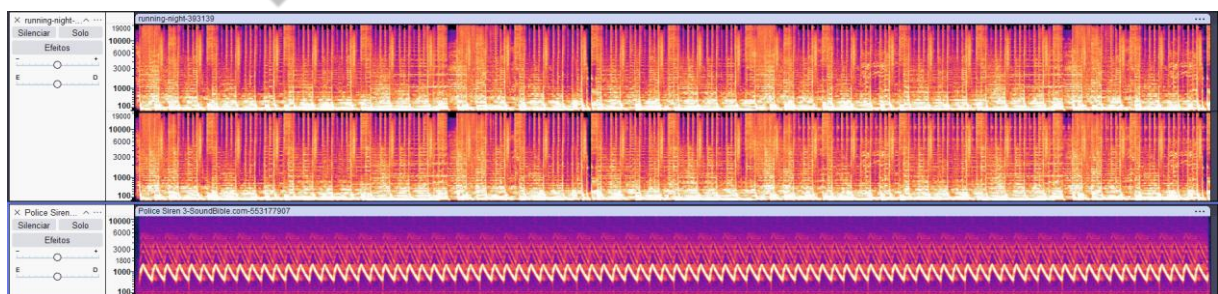


Figura 21 - Espectrograma dos áudios combinados com o Audacity
Fonte: Elaborado pelo autor (2025)

Com o botão de play (ou a barra de espaço), você pode executar o áudio. Perceba que ela tem os dois sons combinados, da música e da sirene. Vamos exportar

o áudio final clicando em “Arquivo” > “Exportar áudio como”. Escolha canal mono e formato WAV, nome do arquivo `musica_com_ruido.wav` conforme mostrado na figura a seguir:

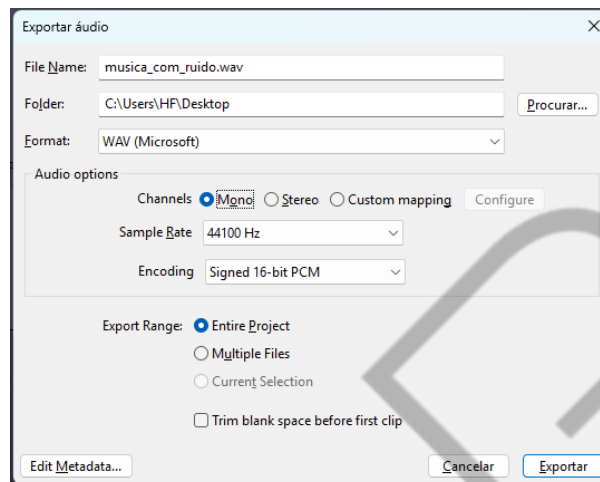


Figura 22 - Exportando áudio combinado com o Audacity
Fonte: Elaborado pelo autor (2025)

Para conferir, abra no Audacity o áudio exportado, você terá algo como:

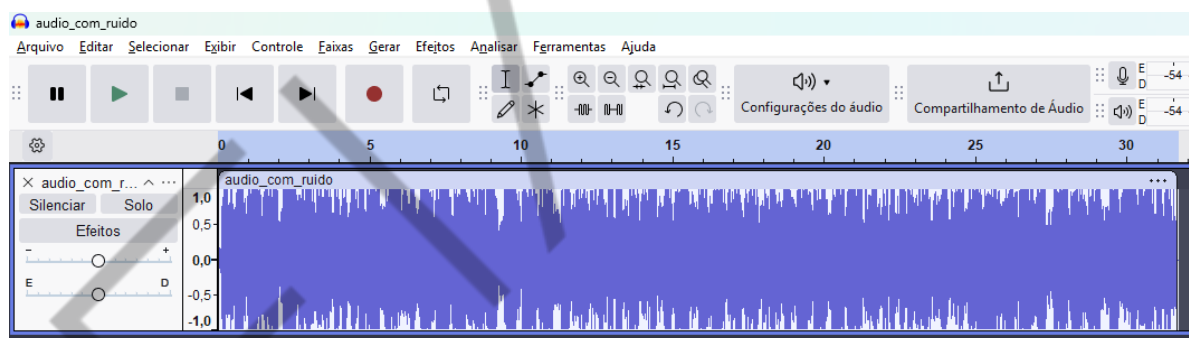


Figura 23 - Resultado áudio combinado com o Audacity no domínio do tempo
Fonte: Elaborado pelo autor (2025)

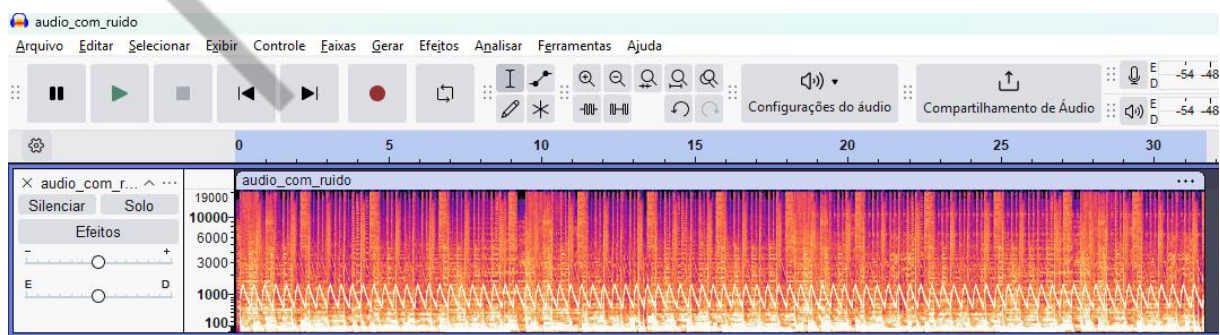


Figura 24 - Resultado áudio combinado com o Audacity, vista de espectrograma
Fonte: Elaborado pelo autor (2025)

Perceba pela comparação das figuras “Espectrograma dos áudios combinados com o Audacity” e “Resultado áudio combinado com o Audacity, vista de espectrograma” que o áudio final só tem um canal e combina a sirene (o dentilhado)

com a música. Tendo esse arquivo, como poderíamos remover apenas o ruído da sirene?

Uma forma simples é aplicar um filtro passa-baixa (LPF) para remoção de agudos, como fizemos no exemplo da figura “Aplicando filtro de Graves e Agudos”. O efeito dentro do Audacity, contudo, não conseguiria atenuar as frequências da sirene propriamente dita. Para tanto, podemos escrever uma função em Python, conforme o código-fonte:

```
from scipy.signal import butter, lfilter
import librosa
import soundfile as sf

def lowpass_filter(data, cutoff=24000, fs=44100,
order=6):
    nyq = 0.5 * fs
    normal_cutoff = cutoff / nyq
    b, a = butter(order, normal_cutoff, btype='low',
analog=False)
    y = lfilter(b, a, data)
    return y

noisy, _ = librosa.load("musica_com_ruido.wav", sr=44100,
mono=True) # Carregando o áudio

noisy_filtered = lowpass_filter(noisy, cutoff=350,
fs=44100, order=6) # Filtrando

sf.write("musica_com_ruido_filtrada_passa_baixa.wav",
noisy_filtered, 44100) # Salvar o áudio filtrado
```

Código-fonte 8 - Aplicando filtro passa-baixa com Python
Fonte: Elaborado pelo autor (2025)

Ouça a música filtrada. O que aconteceu? Não só a sirene ficou muito mais distante, como boa parte da música desapareceu. Podemos, inclusive, ver isso no espectrograma:

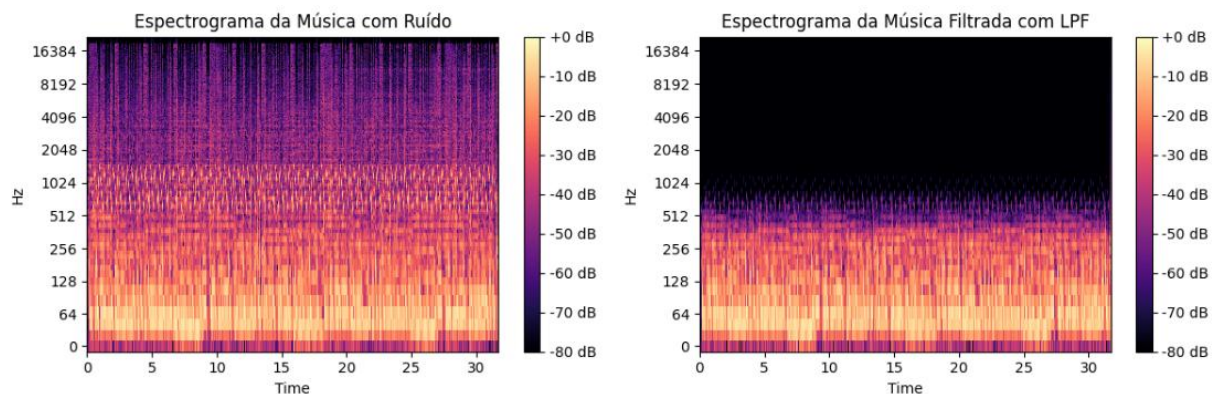


Figura 25 - Resultado áudio combinado filtrado com LPF no Python

Fonte: Elaborado pelo autor (2025)

Para gerar esses gráficos, utilizamos o código-fonte a seguir:

```
import numpy as np
import matplotlib.pyplot as plt
import librosa.display

N_FFT = 2048
HOP_LENGTH = 512
SR = 44100

noisy_spec = np.abs(librosa.stft(noisy, n_fft=N_FFT,
hop_length=HOP_LENGTH))
noisy_filtered_spec = np.abs(librosa.stft(noisy_filtered, n_fft=N_FFT,
hop_length=HOP_LENGTH))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
librosa.display.specshow(
    librosa.amplitude_to_db(noisy_spec, ref=np.max),
    sr=SR, hop_length=HOP_LENGTH,
    x_axis='time', y_axis='log')
plt.title("Espectrograma da Música com Ruído")
plt.colorbar(format='%+2.0f dB')

plt.subplot(1, 2, 2)
librosa.display.specshow(
    librosa.amplitude_to_db(noisy_filtered_spec,
ref=np.max),
    sr=SR, hop_length=HOP_LENGTH,
    x_axis='time', y_axis='log')
plt.title("Espectrograma da Música Filtrada com LPF")
plt.colorbar(format='%+2.0f dB')
plt.tight_layout()
plt.show()
```

Código-fonte 9 - Gerando os espectrogramas da Figura 25

Fonte: Elaborado pelo autor (2025)

Como poderíamos fazer melhor? Treinando uma rede neural artificial end-to-end! Para isso, vamos precisar parear duas amostras de áudio, uma com ruído e uma sem ruído. A ideia é que a rede neural aprenda a mapear o áudio com ruído (entrada) para o áudio sem ruído (saída). Mostraremos durante o treinamento esses pares de amostras (entrada/saída) para que a rede aprenda a remover o ruído. Logo, também iremos precisar da música sem ruído com a mesma duração para servir de saída esperada durante o treino. Para obtê-la, basta deletar o canal da sirene no Audacity e exportar novamente, mantendo o mesmo padrão da figura “Exportando áudio combinado com o Audacity”, porém com o nome de arquivo diferente, como `musica.wav` apenas.

Inicialmente, carregamos os dois arquivos `musica_com_ruido.wav` e `musica.wav` e fazemos um pré-processamento básico, no qual trabalharemos com o sinal no domínio do espectrograma (STFT) escalonado por padronização (StandardScaler). Os dados também serão separados em treino e teste, seguindo a regra 80/20. Isso é feito no código-fonte a seguir:

```
import librosa
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt
import soundfile as sf

# =====
# PARÂMETROS
# =====
SR = 44100
N_FFT = 1024
HOP_LENGTH = 256
WINDOW = 128
TRAIN_RATIO = 0.8
EPOCHS = 200
BATCH_SIZE = 8

# =====
# LOAD AUDIO
# =====
clean, _ = librosa.load("musica.wav", sr=SR, mono=True)
noisy, _ = librosa.load("musica_com_ruido.wav", sr=SR,
mono=True)

min_len = min(len(clean), len(noisy))
```

```

clean = clean[:min_len]
noisy = noisy[:min_len]
# =====
# STFT
# =====
clean_stft = librosa.stft(clean, n_fft=N_FFT,
hop_length=HOP_LENGTH)
noisy_stft = librosa.stft(noisy, n_fft=N_FFT,
hop_length=HOP_LENGTH)

clean_mag = np.abs(clean_stft)
noisy_mag = np.abs(noisy_stft)

# =====
# TRAIN / TEST SPLIT NO TEMPO
# =====
split = int(clean_mag.shape[1] * TRAIN_RATIO)

clean_train = clean_mag[:, :split]
noisy_train = noisy_mag[:, :split]

clean_test = clean_mag[:, split:]
noisy_test = noisy_mag[:, split:]

# =====
# NORMALIZAÇÃO
# =====
mean = noisy_train.mean()
std = noisy_train.std() + 1e-8

clean_train = (clean_train - mean) / std
noisy_train = (noisy_train - mean) / std

clean_test = (clean_test - mean) / std
noisy_test = (noisy_test - mean) / std

# =====
# JANELAS
# =====
def make_windows(noisy, clean, window):
    X, Y = [], []
    for i in range(0, noisy.shape[1] - window, window):
        X.append(noisy[:, i:i+window])
        Y.append(clean[:, i:i+window])
    return np.array(X), np.array(Y)

X_train, Y_train = make_windows(noisy_train, clean_train,
WINDOW)
X_test, Y_test = make_windows(noisy_test, clean_test,
WINDOW)

```

```
# =====
# PYTORCH TENSORS
# =====

X_train = torch.tensor(X_train,
dtype=torch.float32).unsqueeze(1)
Y_train = torch.tensor(Y_train,
dtype=torch.float32).unsqueeze(1)

X_test = torch.tensor(X_test,
dtype=torch.float32).unsqueeze(1)
Y_test = torch.tensor(Y_test,
dtype=torch.float32).unsqueeze(1)

train_loader = DataLoader(
    TensorDataset(X_train, Y_train),
    batch_size=BATCH_SIZE,
    shuffle=True
)

test_loader = DataLoader(
    TensorDataset(X_test, Y_test),
    batch_size=1,
    shuffle=False
)
```

Código-fonte 10 - Preparação dos dados para rede end-to-end
Fonte: Elaborado pelo autor (2025)

Após separar os dados de treinamento e de teste e transformá-los nos tensores finais para o modelo de Rede Neural, vamos de fato criar a rede e treiná-la, conforme o próximo código-fonte:

```
# =====
# MODELO
# =====

class DenoisingAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(32, 16, 3, stride=2,
padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 3, stride=2,
padding=1, output_padding=1)
        )
```



```

def forward(self, x):
    return self.decoder(self.encoder(x))

model = DenoisingAutoencoder()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# =====
# TREINAMENTO
# =====
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for noisy_b, clean_b in train_loader:
        out = model(noisy_b)

        min_f = min(out.shape[2], clean_b.shape[2])
        min_t = min(out.shape[3], clean_b.shape[3])
        out = out[:, :, :min_f, :min_t]
        clean_b = clean_b[:, :, :min_f, :min_t]

        loss = criterion(out, clean_b)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch      {epoch+1}/{EPOCHS}      |      Loss:
{total_loss/len(train_loader):.6f}")

```

Código-fonte 11 - Criação e treinamento da Rede Neural end-to-end
 Fonte: Elaborado pelo autor (2025)

A nossa rede será uma Autoencoder que tentará levar o áudio ruidoso (noisy) para o áudio limpo (clean). Estamos treinando por 200 épocas, o que pode ser considerado um sobreajuste. Como exercício complementar para verificar isso, você pode fazer uma separação de dados em treino/validação/teste para verificar o desempenho nos dados de validação em função das épocas.

Neste momento, testaremos o treinamento executado no código-fonte anterior, pedindo para a rede inferir sobre o conjunto de teste e reconstruindo o áudio de teste gerado:

```
# =====
# INFERÊNCIA NO TESTE
# =====
model.eval()
denoised_test = []

with torch.no_grad():
    for noisy_b, _ in test_loader:
        out = model(noisy_b)

        min_f = min(out.shape[2], noisy_b.shape[2])
        min_t = min(out.shape[3], noisy_b.shape[3])
        out = out[:, :, :min_f, :min_t]

        denoised_test.append(out.squeeze().numpy())

denoised_test = np.concatenate(denoised_test, axis=1)

# =====
# RECONSTRUIR E SALVAR ÁUDIO
# =====
denoised_mag = denoised_test * std + mean
phase_test = np.angle(noisy_stft[:, split:split +
denoised_test.shape[1]])

denoised_stft = denoised_mag * np.exp(1j * phase_test)
denoised_audio = librosa.istft(denoised_stft,
hop_length=HOP_LENGTH)

denoised_audio =
denoised_audio.squeeze().astype("float32")
denoised_audio /= np.max(np.abs(denoised_audio) + 1e-8)

sf.write("musica_denoised_test.wav", denoised_audio, SR)
```

Código-fonte 12 - Inferência e reconstrução do áudio com a rede end-to-end

Fonte: Elaborado pelo autor (2025)

Ouçá a música reconstruída: você ainda pode ouvir a sirene? E outros sinais agudos? Compare com o resultado do uso do filtro passa-baixa LPF, qual é melhor? Na figura “**Resultado áudio filtrado com rede end-to-end**”, é feita uma comparação visual com os gráficos gerados de acordo com o código-fonte:

```

import matplotlib.pyplot as plt
import librosa.display

# Sinais
clean_audio_test = clean[split:split +
len(denoised_audio)]
denoised_audio_test = denoised_audio
noisy_audio_test = noisy[split:split +
len(denoised_audio)]

# Espectrogramas
clean_spec = np.abs(librosa.stft(clean_audio_test,
n_fft=N_FFT, hop_length=HOP_LENGTH))
denoised_spec = np.abs(librosa.stft(denoised_audio_test,
n_fft=N_FFT, hop_length=HOP_LENGTH))
noisy_spec = np.abs(librosa.stft(noisy_audio_test,
n_fft=N_FFT, hop_length=HOP_LENGTH))

plt.figure(figsize=(14, 6))
plt.subplot(2, 3, 1)
plt.plot(noisy_audio_test, color='b', alpha=0.5)
plt.title("Noisy (Teste) - Sinal no tempo")
plt.xlabel("Amostras")
plt.ylabel("Amplitude")
plt.xlim(0,250e3)
plt.subplot(2, 3, 2)
plt.plot(clean_audio_test, color='b', alpha=0.5)
plt.title("Clean (Teste) - Sinal no tempo")
plt.xlabel("Amostras")
plt.ylabel("Amplitude")
plt.xlim(0,250e3)
plt.subplot(2, 3, 3)
plt.plot(denoised_audio_test, color='b', alpha=0.5)
plt.title("Denoised (Teste) - Sinal no tempo")
plt.xlabel("Amostras")
plt.ylabel("Amplitude")
plt.xlim(0,250e3)

plt.subplot(2, 3, 4)
librosa.display.specshow(librosa.amplitude_to_db(noisy_s
pec, ref=np.max),
                        sr=SR, hop_length=HOP_LENGTH,
y_axis='log', x_axis='time')
plt.title("Noisy - Espectrograma")
plt.colorbar(format='%+2.0f dB')
plt.subplot(2, 3, 5)
librosa.display.specshow(librosa.amplitude_to_db(clean_s
pec, ref=np.max),
                        sr=SR, hop_length=HOP_LENGTH,
y_axis='log', x_axis='time')

```

```
plt.title("Clean - Espectrograma")
plt.colorbar(format='%+2.0f dB')
plt.subplot(2, 3, 6)
librosa.display.specshow(librosa.amplitude_to_db(denoise
d_spec, ref=np.max),
                        sr=SR, hop_length=HOP_LENGTH,
y_axis='log', x_axis='time')
plt.title("Denoised - Espectrograma")
plt.colorbar(format='%+2.0f dB')

plt.tight_layout()
plt.show()
```

Código-fonte 13 - Geração dos gráficos de comparação para uso da rede end-to-end
Fonte: Elaborado pelo autor (2025)

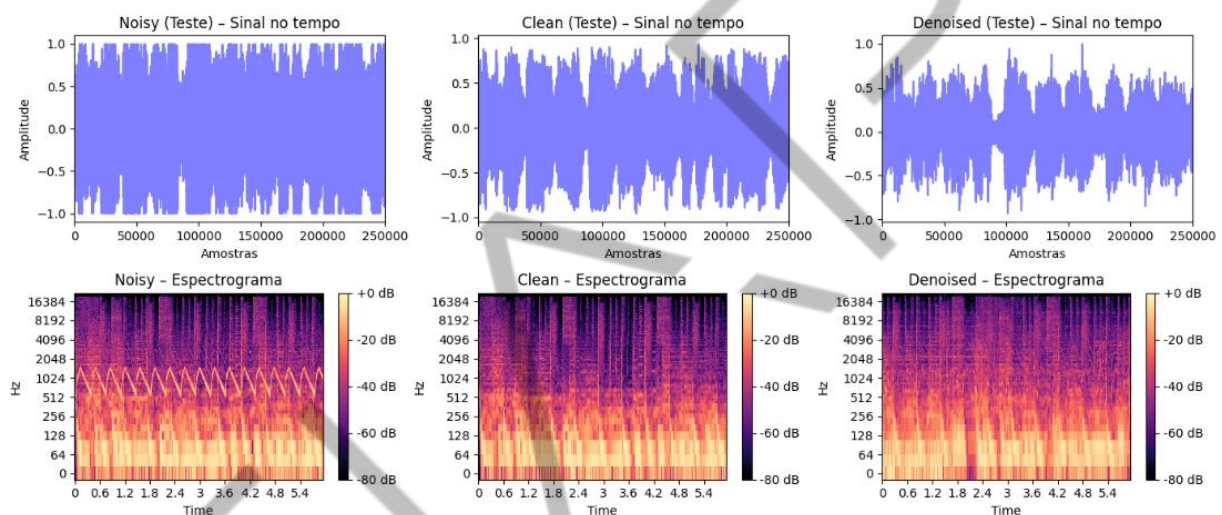


Figura 26 - Resultado áudio filtrado com rede end-to-end
Fonte: Elaborado pelo autor (2025)

IMPORTANTE: Tecnicamente, utilizamos a fase do sinal do próprio conjunto de teste para reconstruir o áudio final, combinando com a magnitude predita pela rede neural. Contudo, para que o algoritmo fosse de fato útil na prática, não poderíamos fazer isso, uma vez que não saberíamos a fase do áudio naquele instante. Uma forma de solucionar esse problema é usar o algoritmo de Griffin-Lim, que permite estimar a fase de um sinal quando apenas o módulo do espectro é conhecido (predito pela rede).

Perceba que no exemplo realizado, a rede aprendeu a tratar apenas a música utilizada. Em outras palavras, ela não deveria ser capaz de remover ruído de outras músicas. Aqui há espaço para a exploração e teste, contudo: será que essa rede seria capaz de remover outro tipo de ruído se fosse usada a mesma música? Você pode testar isso criando um novo áudio combinando com outro ruído. Isso configura a área de estudo da Transferência de Aprendizado, quando um sistema treinado para uma tarefa é utilizado para outra tarefa similar.

Devemos destacar que o objetivo desse exemplo foi mostrar como uma rede neural pode ser construída para levar áudio (entrada) para áudio (saída), caracterizando o que denominamos de modelos end-to-end. Apesar desses modelos dominarem certas aplicações, os modelos baseados em features extraídas ainda são muito importantes, seja por serem, em geral, mais leves computacionalmente falando ou por permitirem interpretação e explicabilidade.

Na próxima seção, exploraremos como Processamento de Áudio e Inteligência Artificial se combinam com o Processamento de Linguagem Natural para criar de fato os Sistemas de Fala e Voz.

3 SISTEMAS DE FALA E VOZ

3.1 Como os seres humanos entendem som e linguagem?

O sistema auditivo humano transforma ondas sonoras em sinais elétricos que o cérebro interpreta como som. O processo começa quando o ouvido externo capta as ondas sonoras e as direciona ao tímpano, fazendo-o vibrar. Essas vibrações são amplificadas pelos ossículos do ouvido médio (martelo, bigorna e estribo) e transmitidas à cóclea no ouvido interno, onde células ciliadas convertem o movimento mecânico em impulsos nervosos. Esses impulsos seguem pelo nervo auditivo até o córtex auditivo no cérebro, permitindo a percepção de frequência, intensidade e localização do som.

Com relação a frequência, nosso ouvido, assim como de outros animais, só é capaz de decodificar uma parte dos sons existentes. Na próxima figura, podemos ver isso com a divisão em três faixas: os **infrassons**, os **sons** (audíveis) e os **ultrassons**.

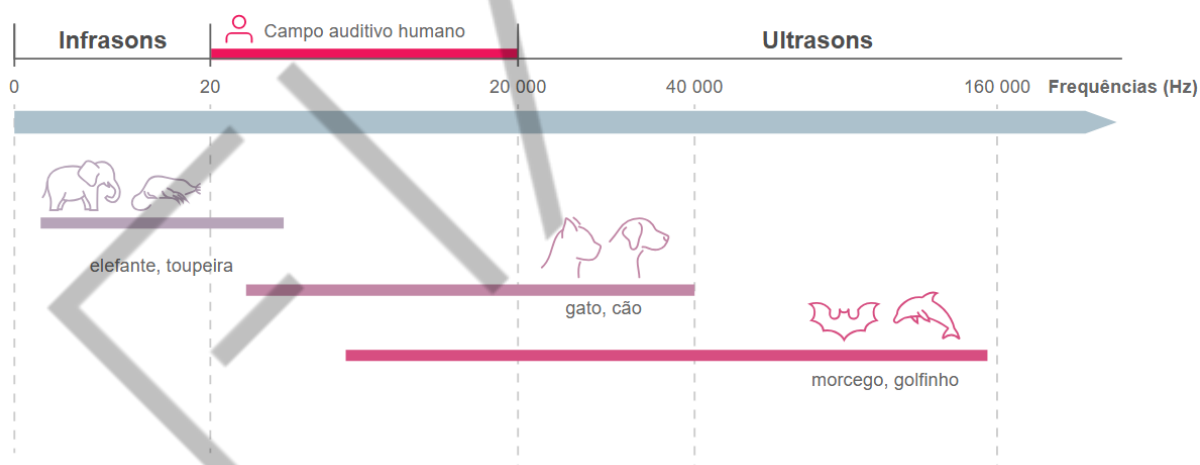


Figura 27 - Espectro audível humano e de outros animais
Fonte: Google Imagens (2025), adaptado por FIAP (2026)

IMPORTANTE: Os espectros são aproximados pela maioria das pessoas: diferenças biológicas sutis podem ocorrer entre indivíduos de uma mesma espécie, seja por fatores genéticos ou ambientais. Crianças, por exemplo, tendem a ouvir sons mais agudos próximos de 24000 Hz, capacidade que vão perdendo conforme envelhecem.

Nos seres humanos, o processo de transformar som em linguagem entendível envolve o cérebro decodificando padrões acústicos em unidades linguísticas reconhecíveis. Primeiro, o córtex auditivo identifica fonemas e entonações; em

seguida, áreas do cérebro responsáveis pelo processamento de linguagem, como a área de Wernicke, associam esses sons a significados, reconhecendo palavras, sintaxe e contexto. O cérebro integra ritmo, prosódia e semântica para reconstruir frases coerentes, permitindo compreender tanto o conteúdo literal quanto as nuances como emoção, ironia ou intenção comunicativa. No quadro a seguir, mostramos os principais elementos linguísticos envolvidos nesse processo de entendimento da fala:

Termo	Definição Linguística
Fonema	Menor unidade sonora que distingue significado em uma língua; função do som para diferenciar palavras.
Prosódia	Conjunto de elementos suprasegmentais — ritmo, acento, entonação e pausas — que estruturam a frase e influenciam o significado.
Ritmo	Padrão de duração (velocidade ou taxa de fala), acentuação e intervalos entre sons ou sílabas, organizando temporalmente a fala. Afeta inteligibilidade e percepção temporal do discurso.
Pausa / Silêncio	Intervalos entre palavras ou frases que segmentam o discurso e indicam limites sintáticos ou pragmáticos.
Entonação	Variação da altura da voz ao falar, transmitindo tipo de frase, emoção ou intenção comunicativa (interrogativa, afirmativa, surpresa). Não muda o significado das palavras isoladas.
Tons lexicais	Variação de altura da voz em sílabas individuais dentro de palavras, mudando o significado lexical da palavra. Exemplo: no mandarim, “mā” (tom alto) = mãe, “mǎ” (tom descendente-ascendente) = cavalo.
Acento	Ênfase em uma sílaba ou palavra, indicando foco, contraste ou mudança de significado; diferente de tom lexical.
Qualidade vocal / Timbre	Características do som da voz (brilho, nasalidade, ressonância) que carregam informações sobre emoção ou identidade do falante.

Quadro 2 - Elementos Linguísticos da Percepção da Fala
Fonte: Elaborado pelo autor (2025)

Quando esses sons são transcritos, os fonemas se refletem em grafemas, combinando-se em sílabas e formando palavras, seguindo as regras da ortografia. A fala pode ser registrada de forma precisa por meio de transcrição fonética, que representa cada som detalhadamente, ou por transcrição ortográfica, que usa a escrita convencional da língua.

Diacríticos e outros sinais gráficos ajudam a refletir nuances de pronúncia ou tonicidade. Dessa forma, há uma continuidade entre a percepção auditiva e a representação escrita: os mesmos elementos que o cérebro decodifica para

compreender a fala servem de base para organizar e codificar a linguagem de maneira visual, permitindo comunicação, aprendizado e registro do conhecimento.

No próximo quadro, resumimos os principais elementos da transcrição fonética e escrita em linguagem natural. Repetimos o termo fonema devido sua importância para ambos os processos, o entendimento da língua no som e a transcrição do som em símbolo.

Termo	Definição Linguística	Papel no processo de escrita/representação
Fonema	Menor unidade sonora distintiva de uma língua.	Serve como base para escolher qual letra ou grupo de letras representará o som.
Grafema	Menor unidade da escrita que representa um fonema (letra ou combinação de letras).	Representa visualmente os sons da fala. Exemplo: /f/ → “f” ou “ph”.
Sílaba	Unidade fonológica formada por um ou mais fonemas, normalmente organizada em ataque, núcleo e coda.	Estrutura básica usada para segmentar palavras na escrita e na leitura. Exemplo: “ca-sa” tem duas sílabas.
Palavra	Unidade mínima de significado em uma língua, formada por uma ou mais sílabas.	Serve como bloco fundamental da escrita, combinando fonemas e sílabas para formar sentido. Exemplo: “casa”, “feliz”.
Ortografia	Conjunto de regras para escrever corretamente palavras e frases.	Define quais grafemas correspondem a quais fonemas e como agrupar sílabas.
Diacríticos	Marcas adicionais nos grafemas que indicam pronúncia, tonicidade, nasalização ou tom.	Ajustam a escrita para refletir nuances sonoras da língua. Exemplo: “ã”, “é”, “ü”.
Transcrição ortográfica	Registro da fala usando a escrita convencional da língua, sem detalhar sons.	Representa as palavras tal como seriam escritas normalmente, útil para documentação ou estudo de textos.
Transcrição fonética	Registro escrito preciso dos sons da fala, usando símbolos como os do Alfabeto Fonético Internacional (AFI).	Permite representar fonemas, entonação e outros detalhes de pronúncia sem ambiguidade.

Quadro 3 - Unidades Linguísticas da Escrita e Transcrição Fonética
Fonte: Elaborado pelo autor (2025)

3.2 Reconhecedores de Fala

Reconhecedores de Fala são sistemas capazes de transformar áudio contendo fala humana em texto escrito. São comumente conhecidos como sistema de Reconhecimento Automático de Fala (ASR, do inglês Automatic Speech Recognition) ou de Fala-para-Texto (STT, do inglês Speech-to-Text). Esses sistemas normalmente

envolvem uma cadeia de processamento que inclui captura e pré-processamento do sinal acústico, extração de features, modelagem acústica e modelagem linguística para estimar a sequência de palavras mais provável.

Abordagens modernas de ASR/STT são predominantemente baseadas em aprendizado profundo, empregando redes neurais convolucionais, recorrentes ou arquiteturas do tipo Transformer, frequentemente treinadas de forma fim a fim (end-to-end). O desempenho desses reconhecedores depende de fatores como qualidade do áudio, ruído ambiental, variações de sotaque, idioma e domínio de aplicação, sendo avaliado por métricas como a Taxa de Erro de Palavra (WER, do inglês Word Error Rate).

3.2.1 ASR Clássico

Um ASR clássico é composto por um modelo acústico, responsável por converter o sinal de áudio em probabilidades de unidades fonéticas, e de um modelo de linguagem, responsável por definir as probabilidades das sequências fonéticas na formação das palavras e das sequências de palavras nas formações de frases.

Para entender como construir um ASR desse tipo, voltemos ao exemplo do dataset Vowel (código-fonte “[Baixando dataset Vowel do OpenML](#)”). Agora, em vez de construirmos um reconhecedor de voz (quem é a pessoa falando), vamos construir um reconhecedor de fala (qual vogal está sendo dita). O próximo código-fonte mostra isso:

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report,
confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Baixar dataset
dataset = fetch_openml(data_id=307, as_frame=True).frame

# 2. Separar features e target
X = dataset.drop(columns=["Speaker_Number", "Sex",
"Class"])
y = dataset["Class"]

# 3. Treino / teste
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)

# 4. Treinar classificador
clf = RandomForestClassifier(n_estimators=100,
random_state=42)
clf.fit(X_train, y_train)

# 5. Avaliação do Desempenho
y_pred = clf.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=clf.classes_, yticklabels=clf.classes_)
plt.title("Matriz de Confusão")
plt.xlabel("Predito")
plt.ylabel("Real")
plt.show()

print(classification_report(y_test, y_pred))
```

Código-fonte 14 - Treinando um reconhecedor de vogais com o dataset Vowel

Fonte: Elaborado pelo autor (2025)

O resultado deste código-fonte é apresentado a seguir:

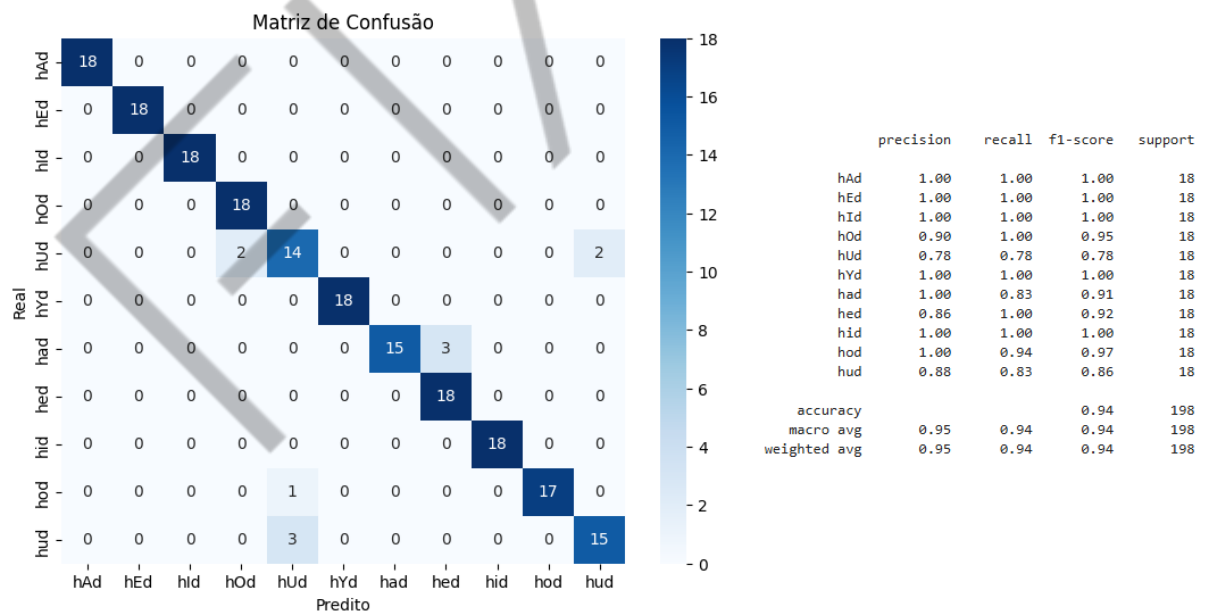


Figura 28 - Resultados do reconhecedor de vogais, saída do Código-fonte 14

Fonte: Elaborado pelo autor (2025)

Perceba que, apesar de estarmos tratando de vogais da língua inglesa, as vogais reconhecidas são marcadas de forma não convencional: hAd, hEd e assim por diante. Por que não apenas A, E, I, O e U? A resposta é que essas são vogais acústicas e não vogais linguísticas dadas pelos grafemas convencionais. Por

exemplo, em português a vogal E pode ser pronunciada como E ou como É. Ou seja, de acordo com esse dataset, existem pelo menos 11 formas diferentes de pronunciar vogais em inglês, mesmo que apenas cinco grafemas sejam utilizados.

Poderíamos expandir nosso reconhecedor de vogais para incluir consoantes também, porém isso geraria um problema: estaríamos apenas entendendo a soletração, ou seja, a vocalização de uma letra solta. Quando duas letras são vocalizadas em conjunto pode ocorrer uma alteração do som final. Por exemplo, na palavra “exemplo” vários sons são alterados: “xe” vira “ze”, o “m” prolonga o som do “e” sem ser pronunciado como “eme”, o “p” modifica-se com o “l” e o “o” virando uma coisa só, “plo”. Que difícil falar e entender fala!

Uma forma de resolver isso é considerar a menor unidade sonora como o fonema que não necessariamente é uma letra solta nem somente uma sílaba. A maioria dos AST Clássicos utiliza um modelo acústico para classificar fonemas, ou seja, transformar áudio em sequências de fonemas mais prováveis.

Uma vez que temos esses fonemas, precisamos convertê-los em grafemas convencionais da escrita. Uma maneira simples é mapear sílabas para fonemas (um fonema pode pertencer a mais de uma sílaba). Como dentro de uma mesma língua podem existir questões de sotaque e entonação, é comum uma mesma sílaba poder ser mapeada para mais de um fonema acústico. Será responsabilidade do modelo de linguagem, além desse mapeamento, realizar a correta concatenação das sílabas. Isso é feito mantendo-se um buffer com as sílabas mais prováveis em cada decodificação.

Considere o diagrama da próxima figura:

Em (a) vemos que o áudio é processado pelo modelo acústico gerando um fonema mais provável, fonema /k/ conforme o Alfabeto Fonético Internacional; esse fonema é então mapeado para uma lista de sílabas prováveis, em ordem de probabilidade (no exemplo, Ka seria mais provável que Ca); essa lista é armazenada em um buffer.

Em (b) vemos que um novo trecho do áudio é processado pelo modelo acústico, gerando o fonema /z/; esse fonema é mapeado para sílabas como Za e As; agora entram as sílabas armazenadas no buffer da janela de processamento anterior, que passam a ser analisadas por um módulo de mapeamento léxico. O mapeamento

léxico consiste em criar palavras que existam na língua analisada. Para isso, o modelo linguístico pode usar uma lista de palavras aceitas (vocabulário) e um modelo n-gram de concatenação de sílabas, para verificar a probabilidade de formação de palavras. Apesar do diagrama parar o exemplo por aqui, é importante notar que outros módulos linguísticos podem ser adicionados, como um n-gram de concatenação de palavras utilizado para formação de frases, servindo como corretor de erros da transcrição das palavras isoladas.

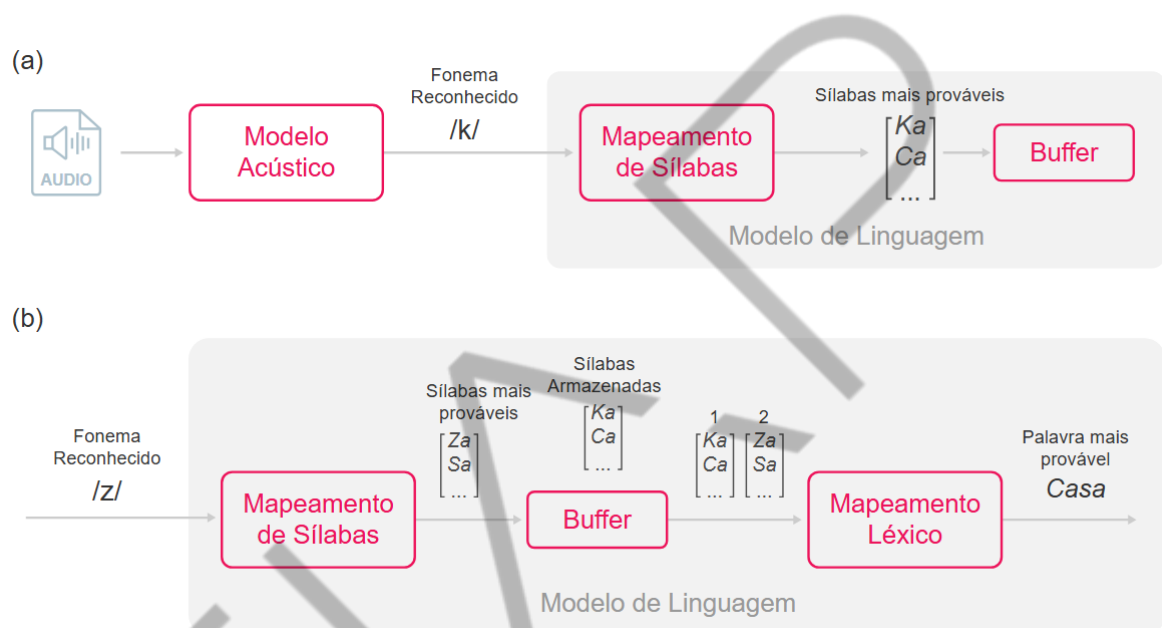


Figura 29 - Diagrama de funcionamento de um ASR clássico
Fonte: Elaborado pelo autor (2025)

3.2.2 Usando um ASR pré-treinado

Como construir um ASR do zero pode ser muito trabalhoso, tanto um sistema clássico em cascata quanto um sistema end-to-end, vamos explorar o uso de ASRs pré-treinados. Para realizar esse exemplo, gravaremos três áudios utilizando o Audacity. Visando calcular a taxa de erro de transcrição, vamos anotar três frases: “olá mundo como você está hoje”, “estamos estudando como funciona um ASR” e “o processamento de áudio aliado ao processamento de linguagem natural permite a criação de aplicações de fala e voz”. Grave cada uma delas em um áudio separado exportando como .wav, canal mono, 44100 Hz de amostragem. Nomeie os arquivos como “teste_1.wav”, “teste_2.wav” e “teste_3.wav” para cada frase, respectivamente.

Vamos instalar alguns módulos do Hugging Face para utilizar um modelo pré-treinado deste site:

```
pip install transformers datasets torchaudio
```

Comando de prompt 2 - Bibliotecas do Hugging Face

Fonte: Elaborado pelo autor (2025)

Teste com a primeira frase utilizando o código-fonte a seguir. Perceba que fizemos um downsampling do áudio original para que o modelo fosse usado. Apesar do modelo não ser totalmente end-to-end, ele utiliza um algoritmo de wav2vec que usa redes neurais artificiais para mapear diretamente séries temporais (sinal de áudio) em vetores de características (vec) aprendido de maneira auto-supervisionada.

```
from transformers import pipeline
import librosa

# Baixa modelo de ASR do Hugging Face
asr = pipeline(
    task="automatic-speech-recognition",
    model="jonatasgrosmann/wav2vec2-large-xlsr-53-
portuguese"
)

# Carrega áudio e já faz downsampling para 16kHz
audio_path = 'teste_1.wav'
audio, sr = librosa.load(audio_path, sr=16000, mono=True)

# Transcrição
resultado = asr(audio)
print(resultado["text"])
```

Código-fonte 15 - Uso de ASR pré-treinado do Hugging Face

Fonte: Elaborado pelo autor (2025)

Também é possível utilizar um modelo pré-treinado em uma nuvem, como Google Cloud, IBM Cloud, AWS e Azure. Na IBM Cloud, procure por Speech to Text na aba de busca, conforme mostra a próxima figura. Perceba que, no caso da imagem, já existe um serviço de STT instanciado, mas, provavelmente, no seu caso, apenas o “Resultados do Catálogo” deve aparecer disponível. Clique sobre o serviço, de forma a ser direcionado para a página mostrada na figura “Instanciando o serviço de ASR/STT na IBM Cloud”. Escolha o plano Lite, com o provedor em Dallas. Você pode mudar o nome do serviço mais abaixo na página. Concorde com os termos de uso e clique em “Criar”.

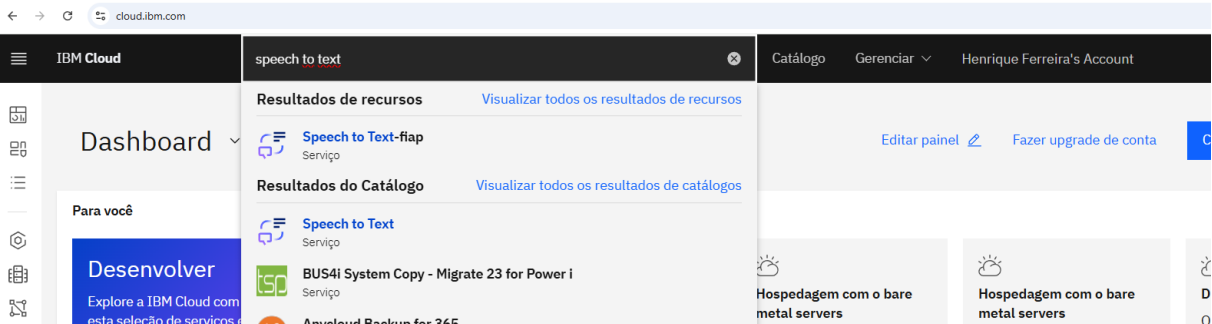


Figura 30 - Criando um ASR na IBM Cloud
Fonte: Elaborado pelo autor (2025)

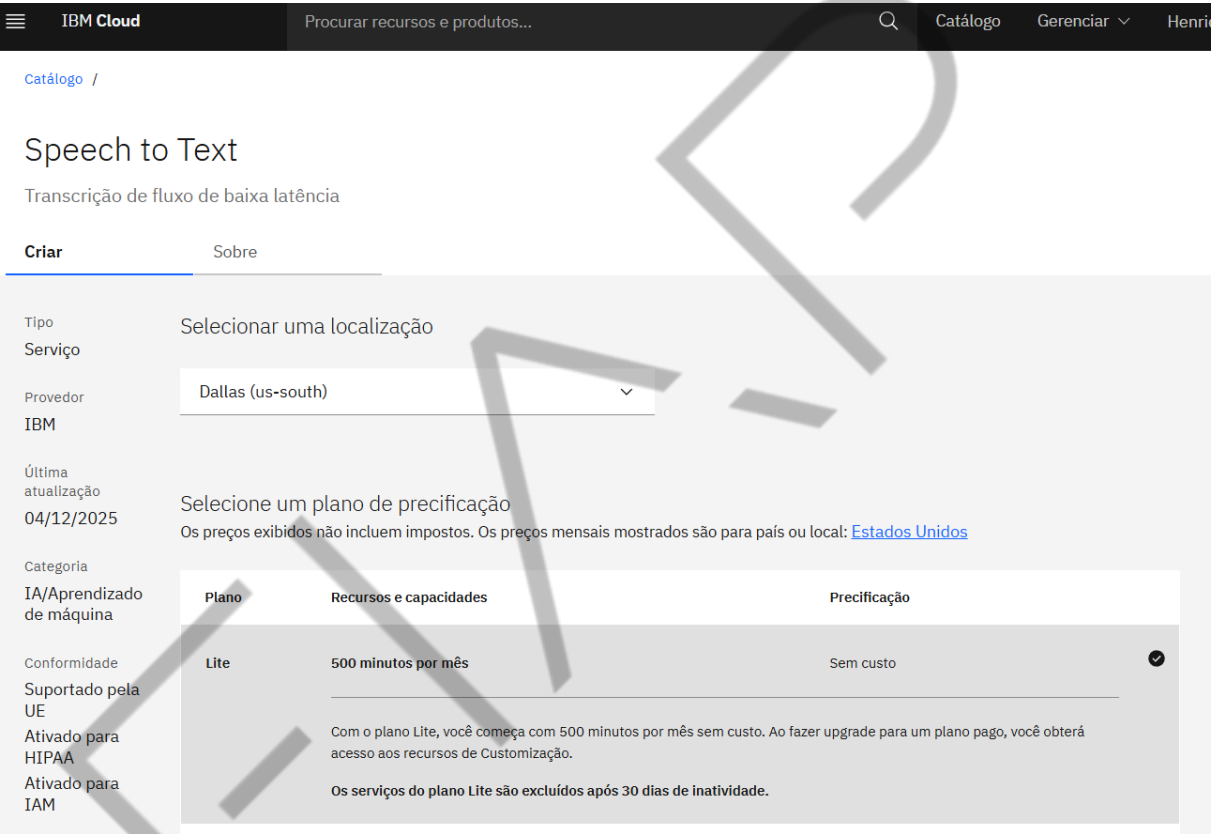


Figura 31 - Instanciando o serviço de ASR/STT na IBM Cloud
Fonte: Elaborado pelo autor (2025)

Uma vez que o serviço estiver instanciado, você deve estar na seguinte página:

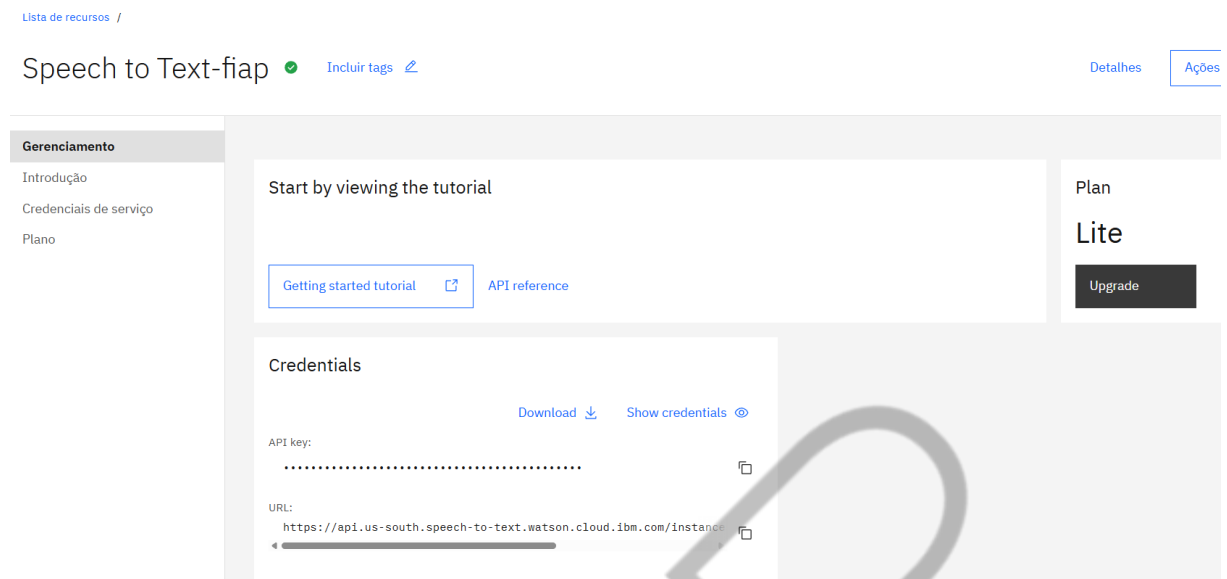


Figura 32 - Serviço de ASR/STT na IBM Cloud pronto para ser usado
Fonte: Elaborado pelo autor (2025)

Para utilizar o serviço, precisamos usar uma API de comunicação com o servidor. Existe suporte para várias linguagens como Java, JavaScript, C# e Python. Ficaremos com o nosso velho conhecido Python para manter a sinergia com os outros exemplos. Instale a API do Python através do pip com:

```
pip install ibm-watson
```

Comando de prompt 3 - Biblioteca do STT e TTS da IBM Cloud
Fonte: Elaborado pelo autor (2025)

Agora, utilize o próximo código-fonte para testar a transcrição do áudio “teste_1.wav”. Perceba que é necessário trocar as credenciais pelas suas credenciais (conforme a figura “Serviço de ASR/STT na IBM Cloud pronto para ser usado”). Além disso, o áudio não será mais processado localmente, como no caso anterior: ele será enviado para os servidores da IBM Cloud para ser processado lá, o que pode gerar latência, mas também facilitar a integração e a disponibilidade. Perceba que, como a resposta vem no formato de dicionário/JSON, ela precisa ser destrinchada.

```
from ibm_watson import SpeechToTextV1
from ibm_cloud_sdk_core.authenticators import IAMAuthenticator

# Credenciais
API_KEY = "SUA_API_KEY_AQUI"
SERVICE_URL = "SUA_SERVICE_URL_AQUI"

# Autenticação
authenticator = IAMAuthenticator(API_KEY)
stt = SpeechToTextV1(authenticator=authenticator)
stt.set_service_url(SERVICE_URL)

# Carregar e enviar áudio
audio_path = "teste_1.wav"
with open(audio_path, "rb") as audio_file:
    resultado = stt.recognize(
        audio=audio_file,
        content_type="audio/wav",
        model="pt-BR_BroadbandModel"
    ).get_result()

# Texto reconhecido
texto = resultado["results"][0]["alternatives"][0]["transcript"]
print(texto)
```

Código-fonte 16 - Uso de ASR/STT da IBM Cloud

Fonte: Elaborado pelo autor (2025)

Agora, testaremos os três áudios gravados com os dois STTs. Primeiramente, vamos instalar o módulo de avaliação para contar a taxa de erro de palavra (WER) com:

```
pip install jiwer
```

Comando de prompt 4 - Bibliotecas para cálculo de WER

Fonte: Elaborado pelo autor (2025)


```
from ibm_watson import SpeechToTextV1
from ibm_cloud_sdk_core.authenticators import IAMAuthenticator
from transformers import pipeline
from jiwer import wer
import librosa
import re

referencias = {
    "teste_1.wav": "olá mundo como você está hoje",
    "teste_2.wav": "estamos estudando como funciona um ASR",
    "teste_3.wav": "o processamento de áudio aliado ao processamento de linguagem natural permite a criação de aplicações de fala e voz"
}

def normalizar(texto):
    texto = texto.lower()
    texto = re.sub(r"^\w\s]", "", texto)
    texto = re.sub(r"\s+", " ", texto).strip()
    return texto

# Pré-treinado do Hugging Face
asr_hf = pipeline(
    task="automatic-speech-recognition",
    model="jonatasgrosman/wav2vec2-large-xlsr-53-portuguese"
)

def transcrever_hf(audio_path):
    audio, sr = librosa.load(audio_path, sr=16000, mono=True)
    resultado = asr_hf(audio)
    return resultado["text"]

# Modelo em Nuvem da IBM
API_KEY = "SUA_API_KEY"
SERVICE_URL = "SUA_SERVICE_URL"

authenticator = IAMAuthenticator(API_KEY)
stt = SpeechToTextV1(authenticator=authenticator)
stt.set_service_url(SERVICE_URL)

def transcrever_ibm(audio_path):
    with open(audio_path, "rb") as audio_file:
        resultado = stt.recognize(
            audio=audio_file,
            content_type="audio/wav",
            model="pt-BR_BroadbandModel"
```

```

        ).get_result()

    return
    resultado["results"][0]["alternatives"][0]["transcript"]

# Loop de teste
resultados = []
for audio, referencia in referencias.items():
    ref = normalizar(referencia)

    hyp_hf = normalizar(transcrever_hf(audio))
    wer_hf = wer(ref, hyp_hf)

    hyp_ibm = normalizar(transcrever_ibm(audio))
    wer_ibm = wer(ref, hyp_ibm)

    resultados.append({
        "audio": audio,
        "WER_HF": wer_hf,
        "WER_IBM": wer_ibm
    })

print(f"\nÁudio: {audio}")
print(f"Referência: {ref}")
print(f"HF: {hyp_hf} | WER = {wer_hf:.2f}")
print(f"IBM: {hyp_ibm} | WER = {wer_ibm:.2f}")

```

Código-fonte 17 - Calculando WER de dois ASR/STT

Fonte: Elaborado pelo autor (2025)

No código-fonte anterior, juntamos os códigos-fonte “**Uso de ASR pré-treinado do Hugging Face**” e “**Uso de ASR/STT da IBM Cloud**” para criar um loop de teste sobre as três frases gravadas. Dependendo da qualidade do seu áudio gravado, seu sotaque e barulhos de fundo capturados, as taxas podem ser diferentes. Na figura a seguir, é apresentado um exemplo de resultado esperado:

```

Áudio: teste_1.wav
Referência: olá mundo como você está hoje
HF: olá mundo como você está hoje | WER = 0.00
IBM: o mundo como você está hoje | WER = 0.17

Áudio: teste_2.wav
Referência: estamos estudando como funciona um asr
HF: estamos estudando como funciona um aesseeerre | WER = 0.17
IBM: estamos estudando como funcionam o sr | WER = 0.50

Áudio: teste_3.wav
Referência: o processamento de áudio aliado ao processamento de linguagem natural permite a criação de aplicações de fala e voz
HF: o processamento de áudio aliado ao processamento de linguagem natural permite a criação de aplicações de fala e voz | WER = 0.05
IBM: o processamento de alto aliado ao processamento de linguagem natural permite criação de aplicações de falas e votos | WER = 0.21

```

Figura 33 - Taxa de erro de transcrição, saída do Código-fonte 17

Fonte: Elaborado pelo autor (2025)

3.3 Como os seres humanos produzem som e linguagem?

O processo de produção da fala em seres humanos envolve a transformação de intenções linguísticas em sinais sonoros articulados. Esse processo inicia-se no cérebro, onde áreas responsáveis pela linguagem, como a área de Broca, planejam a estrutura da fala, selecionando fonemas, ritmo e entonação. Em seguida, comandos motores são enviados aos músculos envolvidos na fonação e articulação. A produção do som ocorre quando o ar expirado dos pulmões passa pela laringe, fazendo vibrar as pregas vocais e gerando a voz.

Na figura a seguir, apresenta-se o espectro de sons produzidos por humanos e outros animais, evidenciando as faixas de frequência associadas à comunicação vocal.

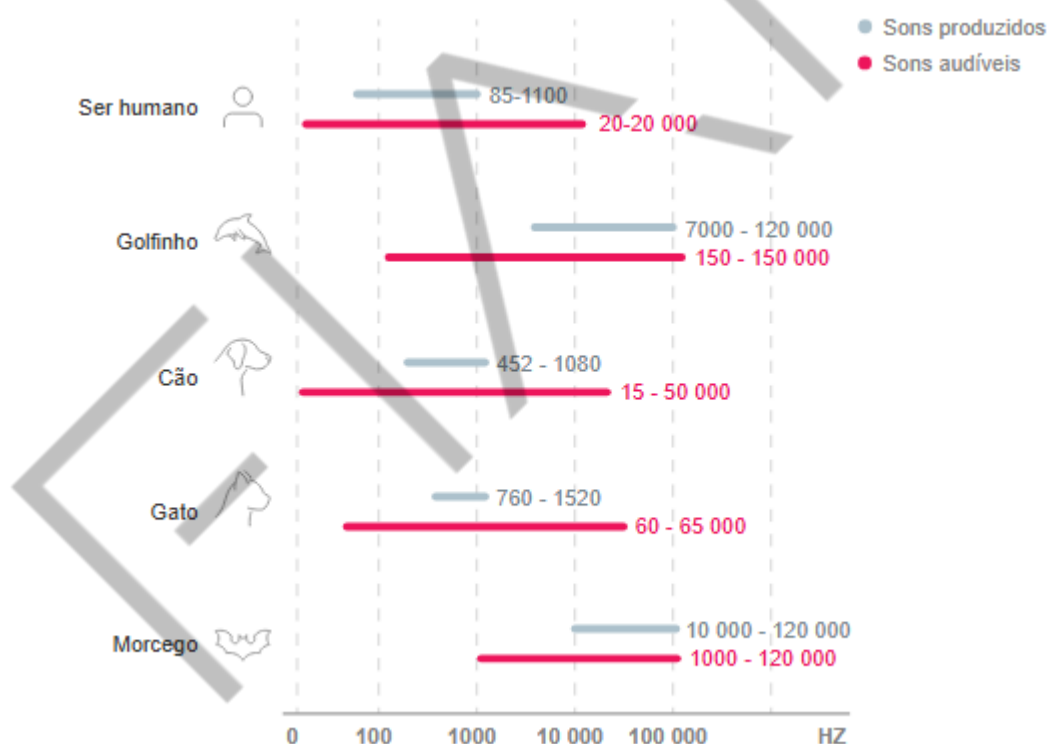


Figura 34 - Espectro de sons produzidos por humanos e de outros animais
Fonte: Google Imagens (2025), adaptado por FIAP (2026)

A frequência e a intensidade do som variam conforme a tensão das pregas vocais e o fluxo de ar. Esse som básico é então moldado pelos articuladores (língua, lábios, dentes, palato e cavidade nasal), que modificam o sinal acústico para formar diferentes fonemas, sílabas e palavras. Dizemos que a boca constitui uma cavidade

ressonante com a língua e lábios, realizando o papel da modulação do sinal base formado na vibração das cordas vocais.

Do ponto de vista linguístico, a fala produzida por esse aparato fisiológico organiza-se em diferentes tipos de sons, principalmente vogais, consoantes e, em muitas línguas, elementos tonais ou prosódicos. As vogais são geradas com o trato vocal relativamente aberto, sendo definidas principalmente pela posição da língua e dos lábios, enquanto as consoantes resultam de obstruções ou constrições do fluxo de ar em pontos específicos do trato vocal. Já os aspectos tonais e prosódicos, como altura, duração e intensidade, dependem do controle das pregas vocais e do ritmo da emissão, sendo fundamentais para diferenciar significados, marcar ênfase ou expressar emoções.

Esses elementos linguísticos estão diretamente ligados ao processo de produção da fala descrito anteriormente: os comandos neurais planejam e coordenam os movimentos articulatórios e o padrão de fonação, transformando intenções comunicativas em unidades sonoras organizadas que constituem a linguagem falada.

3.4 Sintetizadores de Fala

Os sintetizadores de voz, também conhecidos como TTS (Text-to-Speech, texto para fala), buscam reproduzir artificialmente o processo humano de produção da fala e podem ser classificados conforme a técnica utilizada. Os sintetizadores concatenativos baseiam-se na junção de trechos de áudio previamente gravados, como fonemas, sílabas ou palavras, selecionados a partir de grandes bancos de dados e concatenados de acordo com regras linguísticas e prosódicas. Já os sintetizadores baseados em modelos de geração utilizam descrições paramétricas da fala, como frequência fundamental, envelope espectral e duração, para gerar o sinal sonoro a partir de modelos matemáticos do trato vocal e da fonação.

Mais recentemente, os sistemas end-to-end baseados em redes neurais profundas aprenderam diretamente a relação entre texto e áudio, integrando análise linguística, prosódia e geração acústica em um único modelo, produzindo vozes mais naturais, expressivas e adaptáveis, com menor necessidade de regras explícitas ou segmentação manual.

Você pode ver um exemplo de um áudio concatenativo antigo em https://en.wikipedia.org/wiki/File:Fidelity_Chess_Challenger_Voice_speech_output.flac. Perceba a voz metálica e as pausas entre fonemas. Essa é a característica central de um modelo concatenativo. Podemos construir um nós mesmos em português. Para fazer isso, use os dados contidos em https://github.com/ferreirahs/tts_concatenativo e o código-fonte a seguir. Perceba que apenas com alguns poucos áudios gravados de fonemas é possível construir diferentes frases.

```
import librosa
import soundfile as sf
import numpy as np
import json

def sintetizador_voz(frase, dicionario_silabas,
dicionario_fonemas, pasta_fonemas):

    tokens_palavras = frase.split(' ') # Quebra a frase
em palavras
    audio_sintetizado=np.array([]) # Arquivo de audio que
está sendo gerado

    for palavra in tokens_palavras:
        for fonema in dicionario_silabas[palavra]:
            caminho =
pasta_fonemas+'/' +dicionario_fonemas[fonema]
            sinal, sr = librosa.load(caminho, sr=44100)
            audio_sintetizado =
np.concatenate((audio_sintetizado,sinal), axis=0)

    return(audio_sintetizado)

with open('frases_tts.txt', 'r', encoding='utf-8') as
file:
    frases = file.readlines()

    with open('dicionario_silabas.json', 'r', encoding='utf-
8') as file:
        dicionario_silabas = json.load(file)

    with open('dicionario_fonemas.json', 'r', encoding='utf-
8') as file:
        dicionario_fonemas = json.load(file)

    pasta_fonemas = 'audios_fonemas'
```

```
for i, frase in enumerate(frases):
    sentence = frase.replace('\n', '')
    audio_sintetizado = sintetizador_voz(sentence,
dicionario_silabas, dicionario_fonemas, pasta_fonemas)
    sf.write(f'output_audio/a{i}.wav',
audio_sintetizado, samplerate=44100, format='wav')
```

Código-fonte 18 - Criando um TTS concatenativo

Fonte: Elaborado pelo autor (2025)

Poderíamos melhorar nosso sistema adicionando filtros e efeitos entre os áudios concatenados. Se tivermos um modelo acústico de concatenação configurável, podemos utilizar SSML para melhorar o áudio gerado. O SSML (Speech Synthesis Markup Language) é uma linguagem de marcação usada para controlar aspectos da síntese de fala, como pronúncia, pausas, entonação, ritmo e ênfase, dentro de sistemas de TTS, permitindo enriquecer e tornar mais natural a fala sintetizada ao fornecer instruções explícitas ao sintetizador sobre como o texto deve ser vocalizado.

Por exemplo, na frase
< speak>Olá, < break time="500ms"/> tudo bem?</ speak>
o elemento < break> em SSML insere uma pausa de meio segundo na fala sintetizada, tornando a entonação mais natural e correspondente ao “som da vírgula”.

Também podemos utilizar TTS da IBM Cloud. Siga os mesmos passos expressos nas figuras “[Criando um ASR na IBM Cloud](#)”, “[Instanciando o serviço de ASR/STT na IBM Cloud](#)” e “[Serviço de ASR/STT na IBM Cloud pronto para ser usado](#)”, procurando agora por “Text to Speech” na barra de catálogo.

Com o código-fonte a seguir, você pode sintetizar uma frase. Perceba que escolhemos uma voz feminina (Isabela) em português brasileiro.

```
from ibm_watson import TextToSpeechV1
from ibm_cloud_sdk_core.authenticators import IAMAuthenticator

authenticator = IAMAuthenticator("SUA_API_KEY")
tts = TextToSpeechV1(authenticator=authenticator)
tts.set_service_url("SUA_SERVICE_URL")

with open("oi_mundo.wav", "wb") as audio_file:
    audio_file.write(
        tts.synthesize(
            "oi mundo",
            voice="pt-BR_IsabelaV3Voice",
            accept="audio/wav"
```

```
) .get_result().content  
)
```

Código-fonte 19 - Usando TTS da IBM Cloud
Fonte: Elaborado pelo autor (2025)

Existem vários modelos pré-treinados para TTS no Hugging Face que também podem ser explorados. Um famoso modelo end-to-end é o Tacotron, que converte texto diretamente em representações acústicas, geralmente espectrogramas mel, utilizando uma arquitetura encoder-decoder. A geração do sinal de áudio é realizada por um vocoder WaveNet, que transforma essas representações em onda sonora. Esse modelo integra processamento linguístico, prosódia e acústica em um único sistema, produzindo fala natural sem a necessidade de regras explícitas ou concatenação de unidades pré-gravadas.

IMPORTANTE: Vocoder é um algoritmo que analisa e sintetiza a voz humana, permitindo transformar, modular ou gerar sons a partir de sinais de fala.

3.5 Aplicações avançadas

Para encerrar nossa jornada na encruzilhada entre Processamento de Áudio de Processamento de Linguagem Natural, precisamos citar algumas aplicações avançadas.

Vocoders neurais como WaveNet, HiFi-GAN e Parallel WaveGAN permitem manipular atributos da fala sem alterar o texto subjacente. Ao modificar parâmetros como frequência fundamental, envelope espectral e dinâmica temporal, é possível alterar timbre, altura vocal e estilo de fala. Esses mecanismos são utilizados em tarefas de conversão de voz, nas quais a identidade vocal é trocada mantendo o conteúdo semântico, e em clonagem de voz, onde embeddings de locutor condicionam a geração do áudio.

A clonagem de voz tornou viável a geração de fala sintética com alta similaridade perceptual a vozes reais a partir de pequenas amostras de treinamento. Quando combinada a sistemas de TTS ou a modelos de conversão de voz, essa tecnologia permite gerar áudios que reproduzem o timbre e os padrões prosódicos característicos de um indivíduo. Esse mesmo princípio sustenta os chamados deepfakes de voz, nos quais o conteúdo linguístico pode ser produzido ou alterado independentemente da identidade vocal.

No domínio musical, técnicas semelhantes são aplicadas à correção e transformação do sinal. O autotune pode ser interpretado como uma forma controlada de manipulação da frequência fundamental, ajustando a altura da voz para valores discretos associados a uma escala musical. Em versões modernas, esse processo é realizado por modelos baseados em análise espectral e redes neurais, permitindo correções suaves ou efeitos artificiais deliberados, dependendo da parametrização. São usados em TTS concatenativos também na geração de música artificial, para “casar” a voz do cantor que é sintetizada por um modelo com a música instrumental sintetizada por outro.

A geração automática de música estende essas abordagens para estruturas temporais mais longas e múltiplas fontes sonoras. Modelos podem operar sobre representações simbólicas (como MIDI) ou diretamente sobre espectrogramas, utilizando vocoders para reconstrução do sinal. Quando condicionados por texto, esses sistemas utilizam linguagem natural para controlar atributos musicais como estilo, instrumentação e andamento, aproximando a geração sonora de interfaces linguísticas de alto nível.

Você pode testar um sistema desse tipo gerando suas próprias músicas no Suno (<<https://suno.com/>>) ou Beatoven.ai (<<https://www.beatoven.ai/>>), que são plataformas comerciais com créditos gratuitos, ou utilizando modelos pré-treinados do Hugging Face, como o MusicGen da Meta (<<https://huggingface.co/facebook/musicgen-large>>).

REFERÊNCIAS

IBM Cloud. **IBM STT**. Disponível em: <<https://cloud.ibm.com/apidocs/speech-to-text>>. Acesso em: 20 fev. 2026.

IBM Cloud. **IBM TTS**. Disponível em: <<https://cloud.ibm.com/apidocs/text-to-speech>>. Acesso em: 20 fev. 2026.

JUANG, B. H.; RABINER, L. R. **Automatic Speech Recognition** – A Brief History of the Technology Development. [S. l.: s. n.], 2006.

JURAFSKY, Dan; MARTIN, James H. **Speech and Language Processing**. 3. ed. [S. l.: Stanford University, 2025. Disponível em: <<https://web.stanford.edu/~jurafsky/slp3/>>. Acesso em: 20 fev. 2026.

LATHI, B. P. **Sinais e Sistemas Lineares**. [S. l.]: Bookman, 2006.

TURNEY, Peter. **Exploiting Context When Learning to Classify**. [S. l.], 2002. Disponível em: <<https://arxiv.org/abs/cs/0212035>>. Acesso em: 20 fev. 2026.